

ClientIQ Architecture V2.2

Last updated by | Haroun Ahmady | Apr 14, 2026 at 9:08 AM PDT

ClientIQ - System Architecture Document

Farmers and Merchants Bank (FMB)

Document Classification: Internal - Confidential

Customer: Farmers and Merchants Bank

Version: 1.0

ClientIQ - System Architecture Document

Farmers and Merchants Bank (FMB)

Document Classification: Internal - Confidential

Customer: Farmers and Merchants Bank

Version: 1.0

Date: November 13, 2025

Document Owner: Enterprise Architecture & Engineering

Distribution: FMB Technology Leadership, Engineering Teams, Solution Architects

Executive Summary

The ClientIQ platform is an enterprise-grade customer intelligence system deployed on Farmers and Merchants Bank's **on-premise Windows Server infrastructure**. The system provides a unified 360-degree view of customer relationships, accounts, transactions, and household information, consolidating data from FMB's System of Record (SOR) through the Vault data warehouse into a high-performance application database.

Deployment Architecture: - **Infrastructure:** On-premise Windows Server 2019+ cluster - **Database:** Microsoft SQL Server 2019+ (Always On Availability Groups) - **Reverse Proxy:** Nginx for load balancing and SSL termination - **Authentication:** RSA SAML SSO integration - **Environments:** Development, UAT (User Acceptance Testing), Production - **Data Pipeline:** SOR → Vault → Application Database (initial bulk load + daily delta updates)

Key Technology Stack: - Frontend: React 18 + TypeScript + Material-UI - Backend: Express.js + Node.js hosted on Windows Server - Database: Microsoft SQL Server with Full-Text Search - Session Management: SQL Server session store - Authentication: RSA Identity Provider via SAML 2.0

1. Introduction

1.1 System Overview

The ClientIQ platform consolidates customer data from FMB's System of Record through the Vault data warehouse, delivering a comprehensive 360-degree view of customer relationships, accounts, transactions, and

household information. The system is designed specifically for FMB's on-premise Windows Server infrastructure, leveraging Microsoft SQL Server as the primary data store.

Core Capabilities: - **Customer Intelligence:** Unified customer profiles with household relationships, account aggregation, and transaction history - **Customer Data Management:** Customer information viewing with support for individual, business, trust, and estate types - **Account Management:** Comprehensive account viewing with multi-owner support, debit card information, and transaction history - **Search & Discovery:** Intelligent fuzzy search powered by SQL Server Full-Text Search with relevance ranking - **Customer Notes:** Full CRUD operations on customer notes with versioning and audit trails

Technical Foundation: - React 18 + TypeScript single-page application - Express.js REST API layer hosted on Node.js (Windows Server) - Microsoft SQL Server 2019+ with Always On Availability Groups - RSA SAML 2.0 SSO authentication - Nginx reverse proxy for SSL termination and load balancing

1.2 High Level Architecture Overview

[ClientIQ Architecture V2.2.docx](/attachments/ClientIQ%20Architecture%20V2.2-fd26255e-03ef-432b-9b36-f572ff73ac00.docx)

Note: The diagram illustrates key Client IQ components, though it's not exhaustive.

The solution is to be built as a responsive web application that will extend to mobile platforms in the future. It supports Single Sign-On (SSO) for seamless and secure access.

The application architecture is organized into multiple logical layers:

Interface Layer: Web user interface supporting responsive design and secure client interactions.

Application Services Layer: Implements business logic for client data presentation, entitlement enforcement, and user management.

Integration Layer: Provides secure API and service connectivity with JHA core banking and supporting systems.

Data Layer: Consumes curated data from Vault and SPOT sources and supports data engineering functions for performance, enrichment, and governance.

The architecture includes key foundational components to ensure security, reliability, and operational excellence:

Security: SSO integration, TLS encryption, and user entitlements using RBAC and ABAC models.

Observability: Centralized logging, application health checks, and alerting for proactive monitoring.

DevSecOps Enablement: Automated CI/CD pipelines, code scanning (SAST/DAST), and controlled environment promotion for safe and efficient delivery.

1.3 Technical Architecture

Layered Architecture Pattern: The system employs a clear separation of concerns across five layers:

1. **Presentation Layer:** React components with Material-UI, responsive design
2. **API Layer:** RESTful endpoints with versioning
3. **Business Logic Layer:** Service adapters with PII masking, permission checks
4. **Data Access Layer:** SQL Server parameterized queries, stored procedures, connection pooling
5. **Integration Layer:** ETL pipelines (SOR/Vault), SAML authentication

Architectural Patterns: - **DTO Pattern:** Data Transfer Objects with PII masking via customer adapters - **ETL Pipeline:** Scheduled data loads from SOR → Vault → App DB (initial bulk + daily deltas) - **Repository Pattern:** Data access abstraction through SQL Server stored procedures - **Session Management:** SQL Server session store with secure cookie configuration

Technology Strategy: - **Windows Integration:** Node.js hosted on Windows Server with Nginx reverse proxy - **SQL Server Optimization:** Full-Text Search indexes, query optimization, Always On AG - **Scalability:** Horizontal scaling via Windows Network Load Balancing - **Developer Experience:** TypeScript-first development, automated testing, CI/CD

1.4 Security Posture

The platform implements defense-in-depth security controls following banking industry best practices:

Authentication & Authorization: - **Enterprise SSO:** SAML 2.0 integration with RSA Identity Provider - **Session Management:** Secure cookies (HttpOnly, SameSite, Secure), SQL Server session store - **Access Control:** RBAC (Multiple roles and privilege levels) + ABAC (attribute-based permissions) - **Permission Enforcement:** Every API request validated against user permissions

Data Protection: - **In Transit:** TLS 1.3 encryption via Nginx reverse proxy - **At Rest:** SQL Server Transparent Data Encryption (TDE) - **In Use:** PII masking (SSN: --1234, Tax ID: --5678) - **In Logs:** Sensitive data redacted (no SSN, account numbers, passwords)

Network Security: - Nginx reverse proxy for load balancing and SSL termination - Firewall rules restricting access to FMB corporate network - IDS/IPS integration for threat detection

Audit Logging: - Audit trails for customer notes and role management - 7-year retention in SQL Server audit tables

1.5 Architectural Principles

The platform is architected according to enterprise-grade principles ensuring scalability, security, and maintainability:

Principle	Implementation
On-Premise First	Windows Server cluster, SQL Server Always On, on-prem Nginx
Security by Design	Defense-in-depth: TLS 1.3, TDE encryption, RBAC/ABAC, RSA SAML SSO, PII masking
Enterprise Integration	SOR → Vault → App DB pipeline, SSIS ETL workflows
API First	RESTful API-driven architecture with versioning
Data Sovereignty	Role-based permissions, data masking
Regulatory Compliance	Designed to support compliance requirements
Windows Native	Node.js on Windows, SQL Server integration, Active Directory compatibility
High Availability	SQL Server Always On Availability Groups, automatic failover

1.6 Key Non-Functional Requirements

Performance: - Transaction response time: fast response time - Search query response: fast search performance via SQL Server Full-Text Search - Customer profile load: quick loading - Support multiple concurrent FMB staff users - Handle large customer databases with fast search capabilities

Scalability: - Horizontal scaling of application tier via Windows Network Load Balancing - SQL Server read replicas for query distribution - SQL Server session store for stateful session management - Support for enterprise-scale customer databases

Availability: - high availability SLA - SQL Server Always On Availability Groups for automatic failover - Nginx reverse proxy with health checks and failover

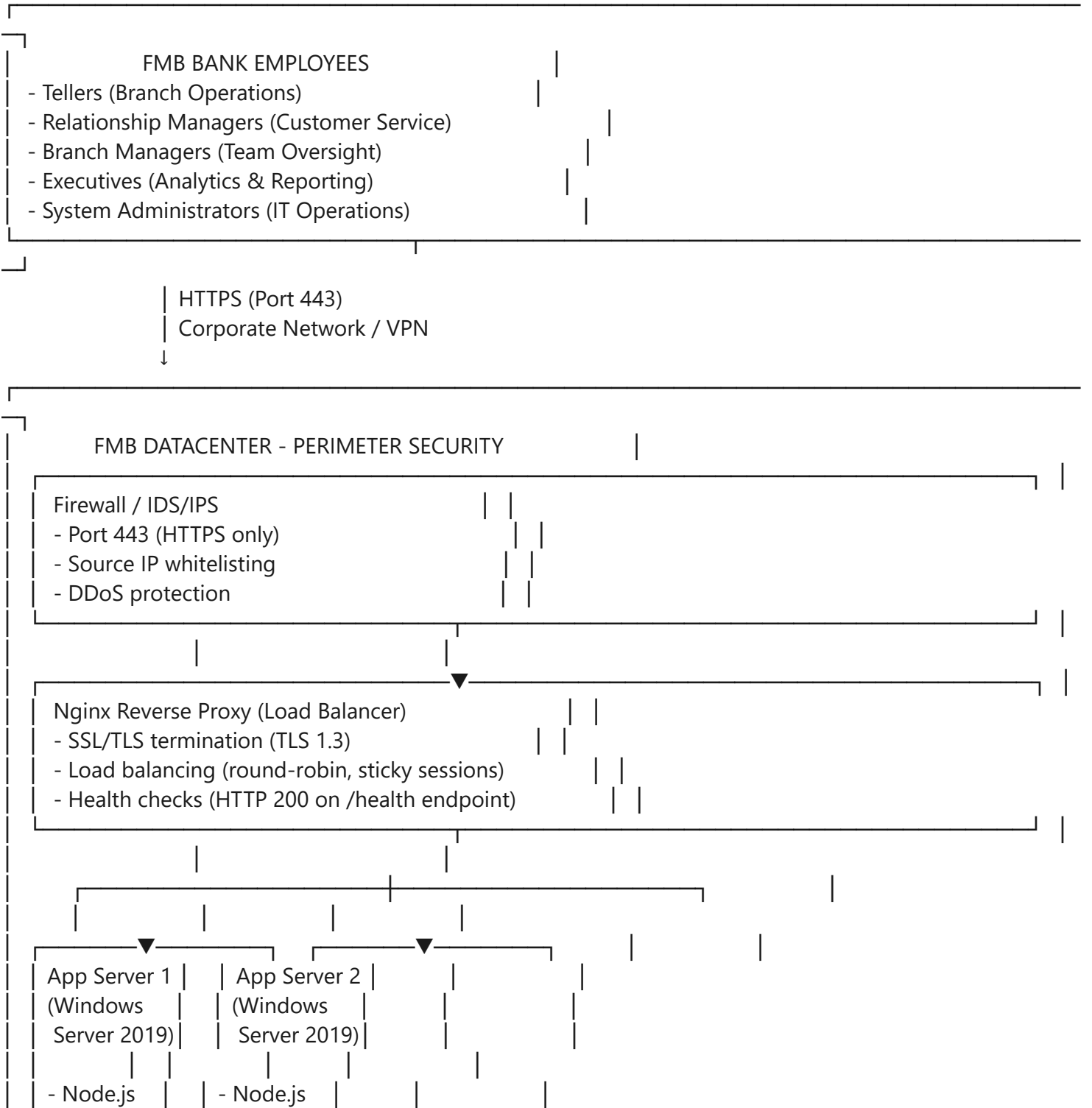
Security: - End-to-end TLS 1.3 encryption - Data masking for PII in UI and logs - Audit logging for notes and role management with 7-year retention

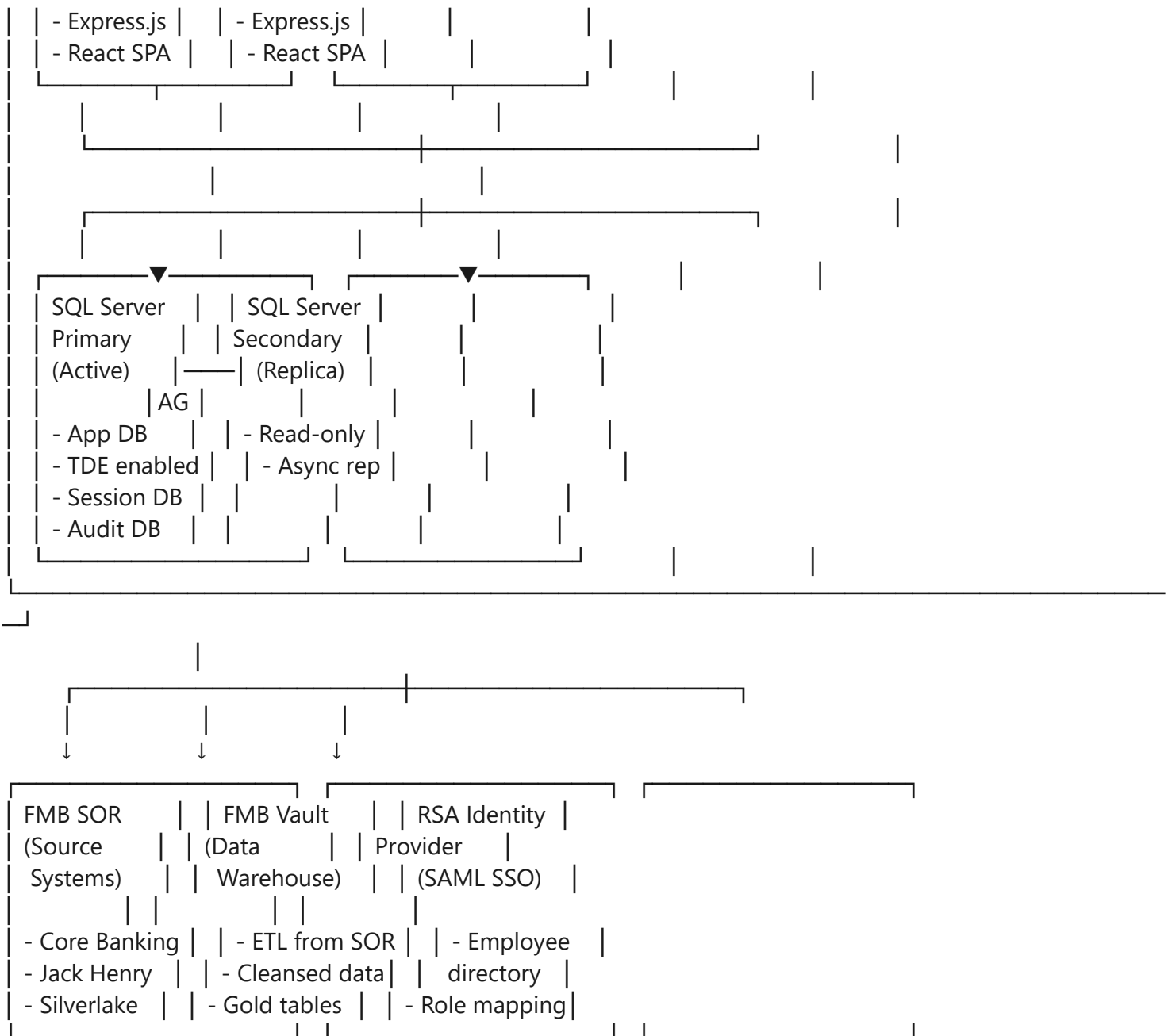
Compliance Standards: - Customer data protected with encryption (TLS 1.3, SQL Server TDE) - Audit logging for notes and role management with 7-year retention - PII masking in UI and logs - Role-based access control for data segregation

2. System Context

2.1 System Context Diagram

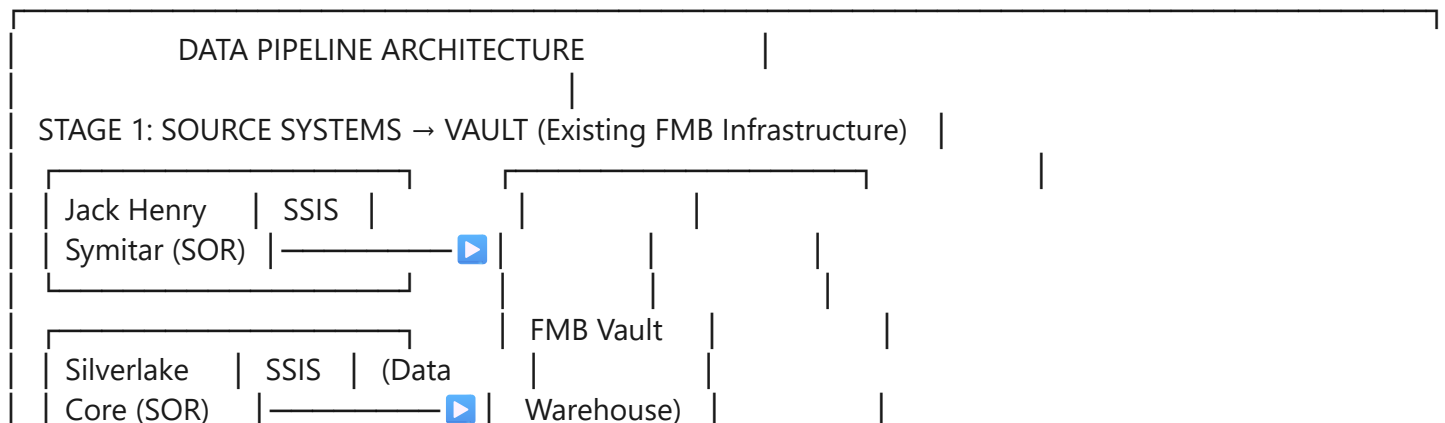
The ClientIQ system operates within FMB’s on-premise datacenter environment:





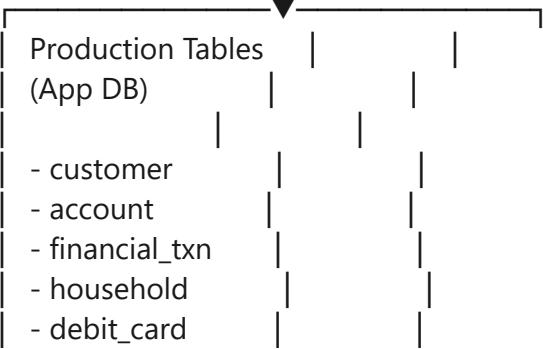
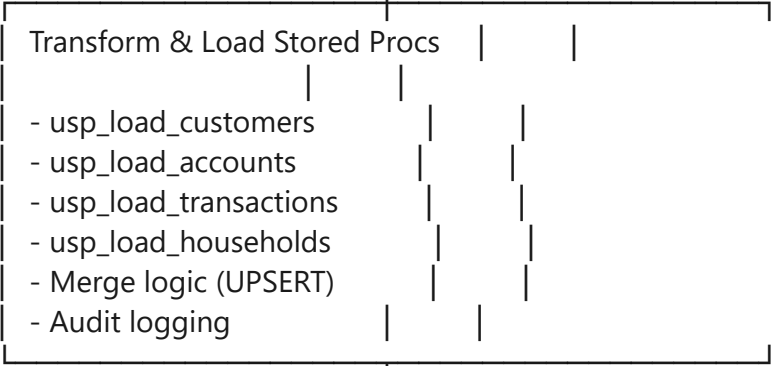
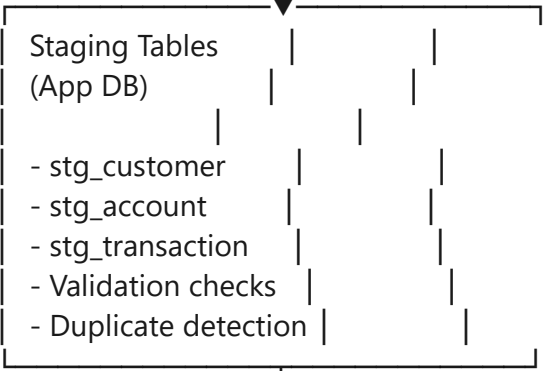
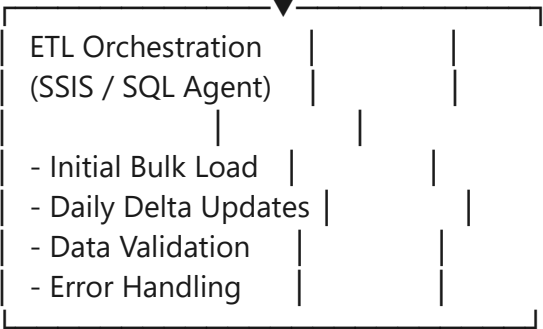
2.2 FMB Data Pipeline Architecture (SOR → Vault → App DB)

The application database is populated through a multi-stage ETL pipeline:





STAGE 2: VAULT → APP DB (ETL Pipeline)



- Full-text indexes

SCHEDULE:

- Initial Load: One-time bulk import (all historical data)
- Delta Loads: Daily during scheduled maintenance window (incremental updates)

Data Load Strategy:

Load Type	Frequency	Volume	Method	Window
Initial Bulk Load	One-time	Large volume of customers and accounts	Full table truncate/reload	Weekend maintenance window
Daily Delta Load	Daily scheduled time	Changed records	Incremental MERGE (UPSERT)	Short maintenance window
Error Handling	Real-time	N/A	Staging table validation, error logging	Alert on failure
Data Validation	Every load	All records	Referential integrity, business rules	Rollback on critical errors

2.3 External System Interfaces

System	Integration Type	Protocol	Data Flow	Authentication
FMB SOR (Jack Henry)	ETL Batch	ODBC/SSIS	Inbound (SOR → Vault)	Windows Auth
FMB SOR (Silverlake)	ETL Batch	ODBC/SSIS	Inbound (SOR → Vault)	Windows Auth
FMB Vault	ETL Batch	SQL Server Link	Inbound (Vault → App DB)	Windows Auth
RSA Identity Provider	SAML 2.0	HTTPS/XML	Bidirectional (auth)	X.509 Certificate

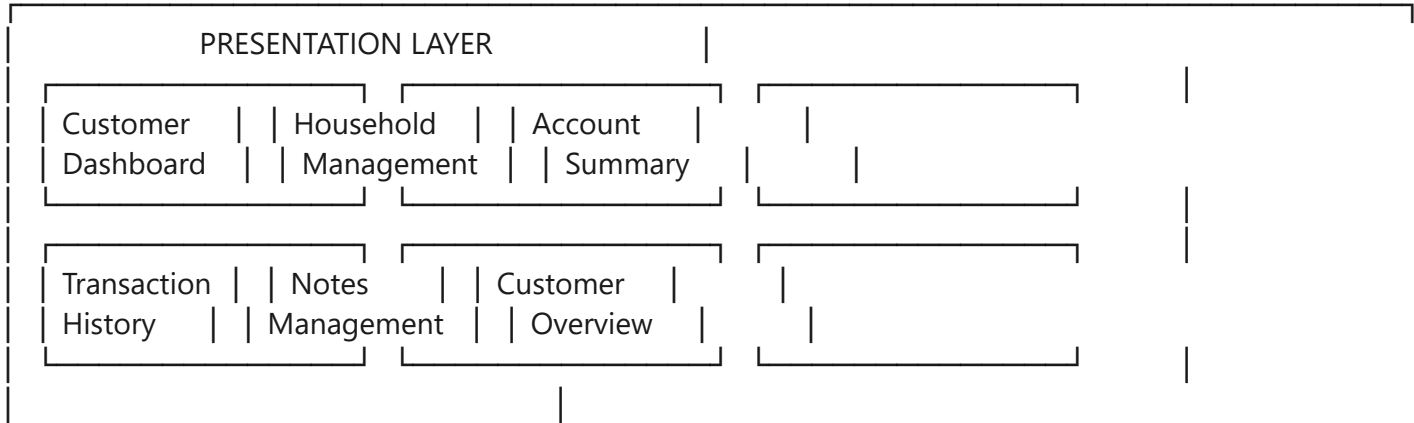
2.4 Stakeholder Analysis

Stakeholder Group	Primary Needs	System Interaction	Success Metrics
Tellers	Fast customer lookup, transaction viewing	Read-only customer profiles, account balances	Fast customer lookup
Relationship Managers	Comprehensive customer intelligence, notes	Full CRUD on customer notes, household management	360° view completeness
Branch Managers	Team oversight, customer service management	All RM capabilities + team reports	High system usage, customer satisfaction
Executives	Strategic analytics, portfolio insights	Dashboards, executive reports (read-only)	Strategic insights availability
System Administrators	User management, system configuration	Full administrative access, role assignment	High system availability
IT Operations	Monitoring, incident response	Infrastructure management, log analysis	MTTR Quick

3. Logical Component Architecture

3.1 Component Structure and Responsibilities

The platform follows a **layered architecture pattern** with clear separation of concerns:



Technology: React 18, TypeScript, Material-UI, shadcn/ui
State Management: TanStack React Query (server state)
Routing: Wouter (SPA routing)
Forms: React Hook Form + Zod validation

↕↕ HTTPS/JSON

API LAYER

API Gateway (Express.js on Windows Server)
- Route definitions (/api/customers, /api/accounts...)
- Request validation (Zod schemas)
- Error handling (standardized error responses)

Authentication Middleware
- SAML 2.0 SSO (npm saml + RSA IdP)
- Session validation (SQL Server session store)
- Employee lookup/upsert from SAML assertions

Authorization Middleware
- RBAC permission checks (role-based)
- ABAC conditional evaluation (attribute-based)
- Privilege level verification

↕↕

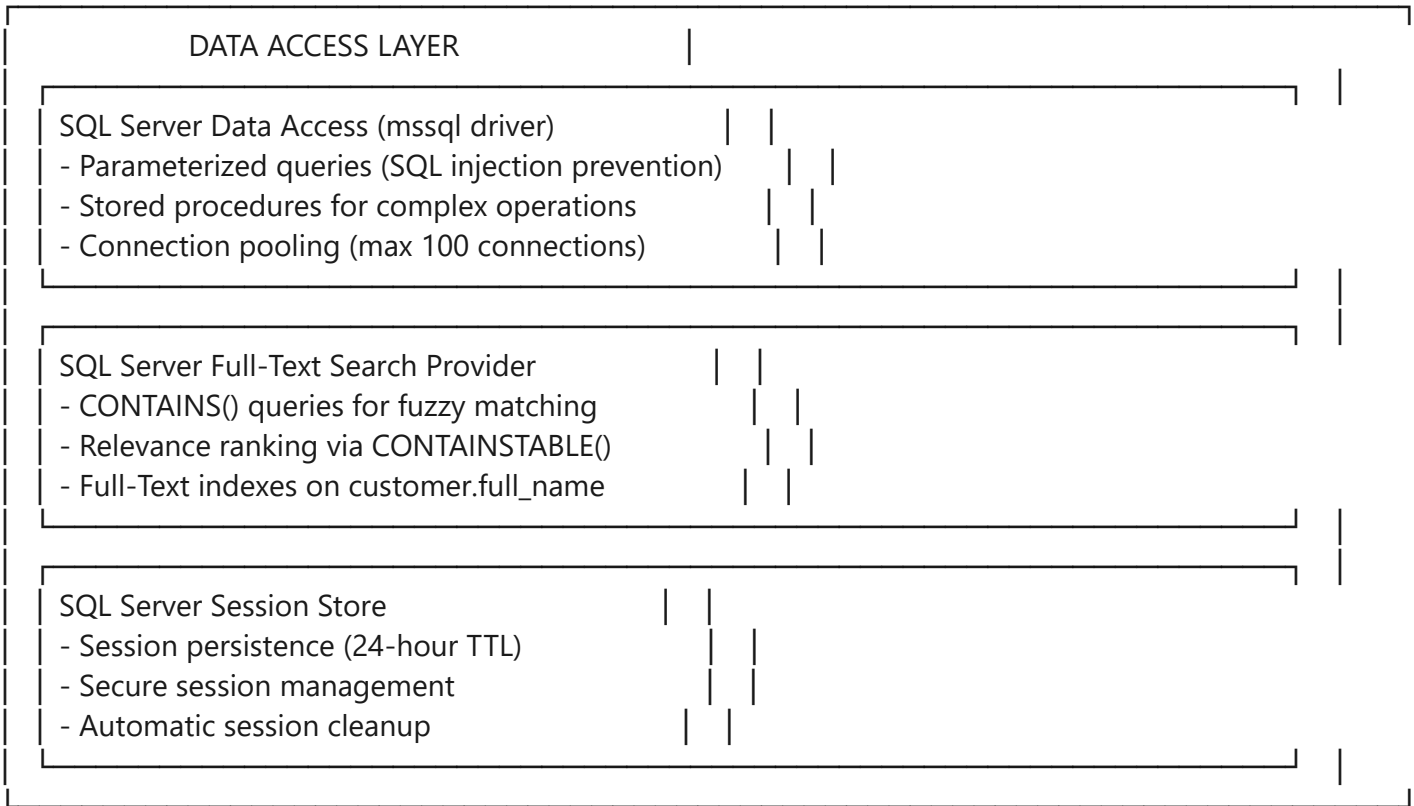
BUSINESS LOGIC LAYER

Customer Adapter	Account Service	Household Service
- PII masking	- Balance calc	- Aggregation
- DTO mapping	- Owner lookup	- Members

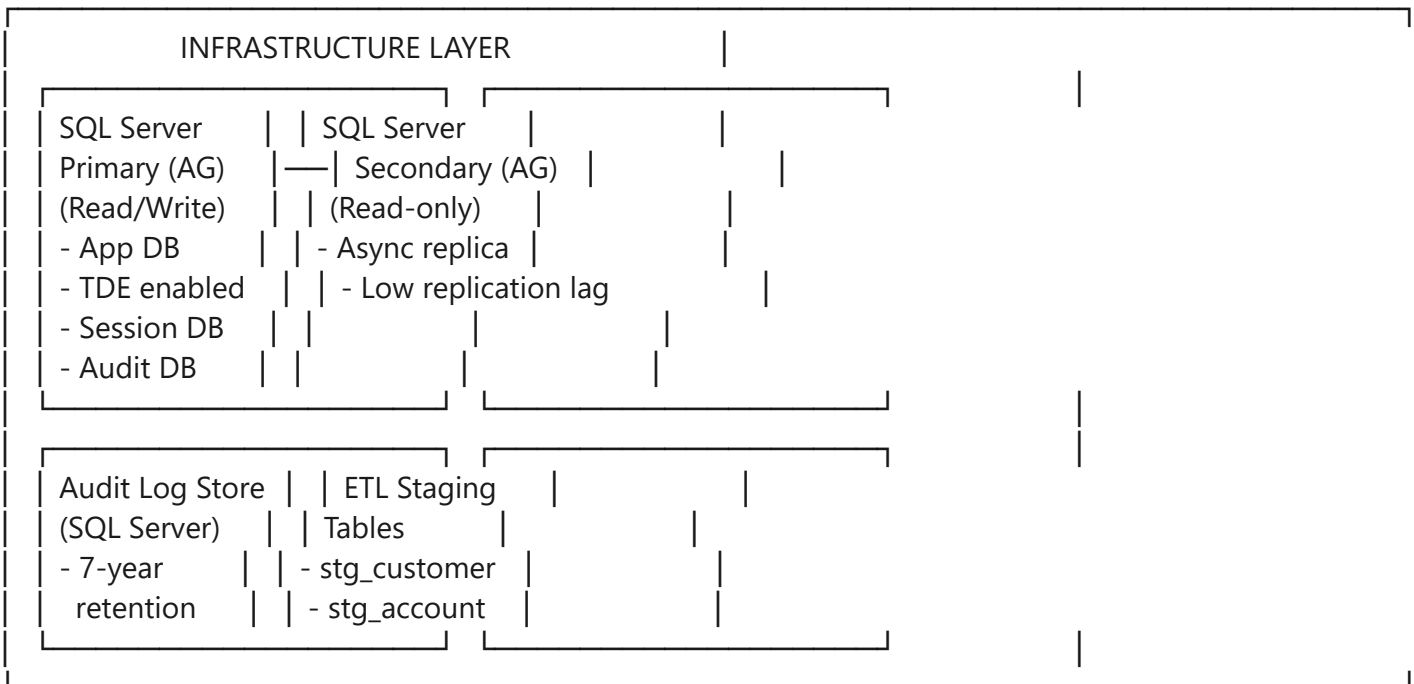
Transaction Service	Search Service	Data Adapter
- Category map	- FTS query	- Field map
- Balance run	- Relevance	- Validation

Permission Service	Note Service
- ABAC eval	- Versioning
- Audit log	- Soft delete

↓↑



↓↑



4. Data Architecture and Information Management

4.1 Data Storage Strategy

SQL Server Primary Database: - **Primary Database:** Microsoft SQL Server 2019+ with Always On Availability Groups - **Read Replicas:** Secondary AG replica for query distribution (read-only) - **Session Store:** SQL Server

session table for stateful session management - **Audit Store:** SQL Server audit tables for 7-year regulatory retention - **Staging Tables:** ETL staging area for data validation before production load

Data Flow Patterns: 1. **Write Operations:** All writes to SQL Server primary (AG) 2. **Read Operations:** Distributed across primary and read replica 3. **Session Management:** SQL Server session store with automatic cleanup 4. **Audit Logs:** Write-through to audit tables (all user actions) 5. **Search Queries:** Optimized via SQL Server Full-Text Search indexes 6. **ETL Loads:** Vault → Staging → Production tables (daily 2:00 AM)

4.2 SQL Server Full-Text Search Configuration

Full-Text Catalog and Index:

```
-- Create Full-Text Catalog
CREATE FULLTEXT CATALOG ftCatalog_CustomerSearch AS DEFAULT;

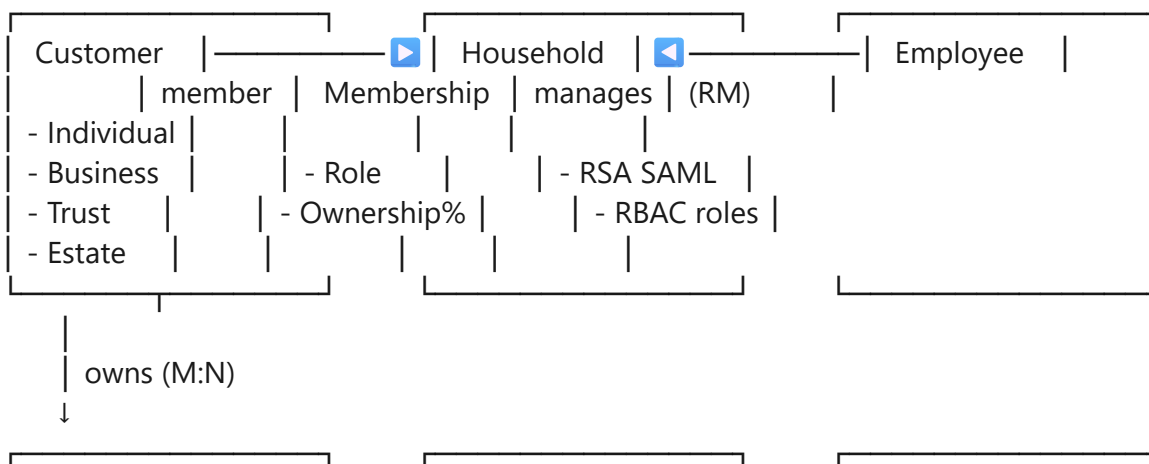
-- Create Full-Text Index on customer table
CREATE FULLTEXT INDEX ON customer(full_name)
KEY INDEX PK_customer_customer_id
ON ftCatalog_CustomerSearch
WITH CHANGE_TRACKING AUTO;

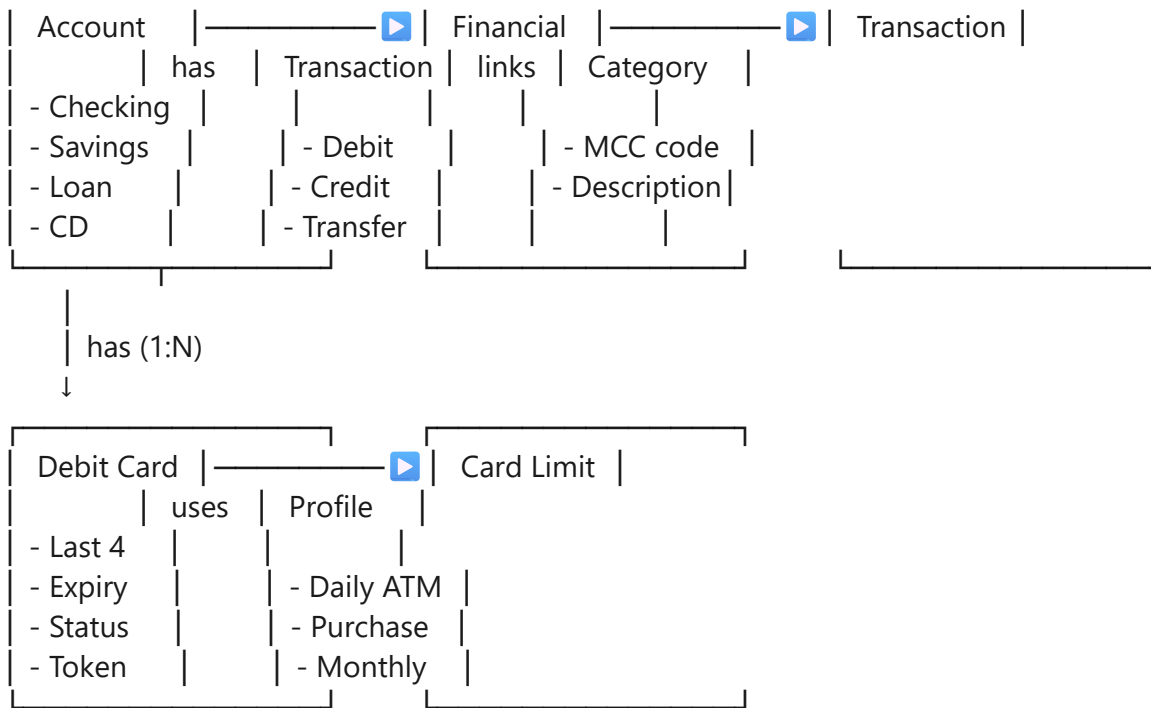
-- Search query example (CONTAINS)
SELECT customer_id, full_name, customer_type
FROM customer
WHERE CONTAINS(full_name, '"John*' OR "Smith*"' )
ORDER BY customer_id;

-- Search query with relevance ranking (CONTAINSTABLE)
SELECT c.customer_id, c.full_name, c.customer_type, ft.RANK
FROM customer c
INNER JOIN CONTAINSTABLE(customer, full_name, '"John Smith"') AS ft
ON c.customer_id = ft.[KEY]
ORDER BY ft.RANK DESC;
```

4.3 Core Data Models

Entity Relationship Overview:





4.4 Database Schema Highlights (SQL Server)

Customer Table:

```

CREATE TABLE customer (
    customer_id BIGINT IDENTITY(1,1) PRIMARY KEY,

    -- Individual customer fields
    first_name NVARCHAR(100),
    last_name NVARCHAR(100),
    middle_name NVARCHAR(100),
    date_of_birth DATE,

    -- Business customer fields
    business_name NVARCHAR(200),
    dba_name NVARCHAR(200),

    -- Unified search field (always populated)
    full_name NVARCHAR(200) NOT NULL,

    -- Identification (masked in UI)
    tax_identifier NVARCHAR(20) UNIQUE,
    government_id NVARCHAR(50),

    -- Classification
    customer_type NVARCHAR(20) NOT NULL DEFAULT 'individual',
    customer_status NVARCHAR(20) DEFAULT 'active',

    -- Compliance
    kyc_status NVARCHAR(20),
  
```

```

risk_rating NVARCHAR(20),

-- Flags
is_employee BIT DEFAULT 0,
vip_customer BIT DEFAULT 0,
is_deceased BIT DEFAULT 0,

-- Core banking integration
jack_henry_cif_number NVARCHAR(20),
silverlake_customer_id NVARCHAR(20),

-- Audit
created_at DATETIME2 DEFAULT GETUTCDATE(),
updated_at DATETIME2 DEFAULT GETUTCDATE(),

-- Constraint: conditional names based on customer type
CONSTRAINT CK_customer_name_type CHECK (
    (customer_type = 'individual' AND first_name IS NOT NULL AND last_name IS NOT NULL) OR
    (customer_type IN ('business', 'trust', 'estate') AND business_name IS NOT NULL)
);

-- Full-Text Index for search
CREATE FULLTEXT INDEX ON customer(full_name)
KEY INDEX PK_customer_customer_id
WITH CHANGE_TRACKING AUTO;

-- Indexes for performance
CREATE INDEX idx_customer_tax_id ON customer(tax_identifier);
CREATE INDEX idx_customer_status ON customer(customer_status);
CREATE INDEX idx_customer_type ON customer(customer_type);

```

SQL Server Session Store:

```

CREATE TABLE sessions (
    session_id NVARCHAR(255) PRIMARY KEY,
    session_data NVARCHAR(MAX),
    expires_at DATETIME2,
    created_at DATETIME2 DEFAULT GETUTCDATE()
);

-- Index for session cleanup
CREATE INDEX idx_sessions_expires ON sessions(expires_at);

-- Automatic cleanup stored procedure
CREATE PROCEDURE usp_cleanup_expired_sessions
AS
BEGIN
    DELETE FROM sessions

```

```
WHERE expires_at < GETUTCDATE();  
END;  
GO
```

4.5 ETL Staging Tables (Vault → App DB)

Staging Table Example:

```
-- Staging table for customer data from Vault  
CREATE TABLE stg_customer (  
    load_id INT, -- ETL batch identifier  
    source_system NVARCHAR(50), -- 'JackHenry' or 'Silverlake'  
    customer_id_src NVARCHAR(50), -- Source system customer ID  
  
    -- Customer data fields (mirroring production table)  
    first_name NVARCHAR(100),  
    last_name NVARCHAR(100),  
    business_name NVARCHAR(200),  
    full_name NVARCHAR(200),  
    tax_identifier NVARCHAR(20),  
    customer_type NVARCHAR(20),  
  
    -- Data quality flags  
    is_valid BIT DEFAULT 1,  
    validation_errors NVARCHAR(MAX),  
  
    -- Audit  
    loaded_at DATETIME2 DEFAULT GETUTCDATE()  
);  
  
-- Stored procedure: Load customers from staging to production  
CREATE PROCEDURE usp_load_customers  
    @load_id INT  
AS  
BEGIN  
    SET NOCOUNT ON;  
  
    -- Validate staging data  
    UPDATE stg_customer  
    SET is_valid = 0,  
        validation_errors = 'Missing required fields'  
    WHERE load_id = @load_id  
        AND (full_name IS NULL OR customer_type IS NULL);  
  
    -- MERGE (UPSERT) into production table  
    MERGE INTO customer AS target  
    USING (  
        SELECT * FROM stg_customer  
        WHERE load_id = @load_id AND is_valid = 1
```



```

) AS source
ON target.jack_henry_cif_number = source.customer_id_src
   OR target.silverlake_customer_id = source.customer_id_src

WHEN MATCHED THEN
  UPDATE SET
    first_name = source.first_name,
    last_name = source.last_name,
    full_name = source.full_name,
    updated_at = GETUTCDATE()

WHEN NOT MATCHED THEN
  INSERT (first_name, last_name, business_name, full_name,
          tax_identifier, customer_type, jack_henry_cif_number)
  VALUES (source.first_name, source.last_name, source.business_name,
          source.full_name, source.tax_identifier, source.customer_type,
          source.customer_id_src);

-- Log results
DECLARE @inserted INT = @@ROWCOUNT;
PRINT 'Customers loaded: ' + CAST(@inserted AS NVARCHAR);

END;
GO

```

4.6 Data Retention and Privacy

Data Type	Retention Period	Deletion Method	Regulatory Basis
Customer PII	7 years post-account closure	Hard delete after retention	GLBA, state banking laws
Account Data	7 years post-closure	Archival then purge	Reg CC, Reg E, UCC
Transactions	7 years	Archival then purge	Regulatory requirements
Debit Card Data	7 years post-cancellation	Immediate tokenization, purge after retention	PCI DSS
Audit Logs	7 years minimum	Indexed storage in SQL Server	Regulatory requirements
Customer Notes	7 years or indefinite (legal hold)	Soft delete (is_deleted flag)	Internal policy
Session Data	Standard session duration	Automatic cleanup via stored procedure	Security policy
SAML Assertions	Regular interval	Rolling deletion	SAML spec

PII Handling: - **At Rest:** SQL Server TDE (Transparent Data Encryption) for database files - **In Transit:** TLS 1.3 for all network communication (Nginx reverse proxy) - **In Use:** PII fields masked in UI (SSN: --1234, Tax ID: --5678) - **In Logs:** PII fields redacted (never log SSN, account numbers, passwords)

5. Technology Stack and Integration Patterns

5.1 Technology Stack Specification for FMB

Frontend Technologies:

Category	Technology	Version	Purpose
UI Framework	React	18.3.1	Component-based UI
Language	TypeScript	5.6.3	Type-safe development
Build Tool	Vite	5.4.19	Fast dev server
UI Library	Material-UI (MUI)	7.3.2	Enterprise components
UI Components	shadcn/ui (Radix)	Latest	Accessible primitives
Styling	Tailwind CSS	3.4.17	Utility-first CSS
State Management	TanStack React Query	5.60.5	Server state caching
Routing	Wouter	3.3.5	SPA routing
Form Handling	React Hook Form	7.55.0	Performant forms
Validation	Zod	3.24.2	Schema validation
Date Handling	date-fns	3.6.0	Date manipulation
Charts	Recharts	2.15.2	Data visualization

Backend Technologies:

Category	Technology	Version	Purpose
Runtime	Node.js	20.x (LTS)	JavaScript runtime
Framework	Express.js	4.21.2	Web framework
Language	TypeScript	5.6.3	Type-safe backend
Database Driver	mssql	12.0.0	SQL Server driver
SAML Authentication	npm saml	Latest	SAML 2.0 SSO
Session Store	connect-mssql-v2	Latest	SQL Server sessions
Password Hashing	bcryptjs	3.0.2	Secure password storage
Validation	Zod	3.24.2	Request validation
Process Manager	tsx	4.19.1	TypeScript execution

Database Technologies:

Category	Technology	Version	Purpose
Primary Database	SQL Server	2019+	Relational database
Search	Full-Text Search	Built-in	Fuzzy text matching
High Availability	Always On AG	Built-in	Automatic failover
Encryption	TDE	Built-in	Data at rest encryption
Session Management	SQL Server Table	Built-in	Session persistence
ETL	SSIS / SQL Agent	2019+	Data pipeline

Infrastructure Technologies:

Category	Technology	Purpose
Operating System	Windows Server 2019+	App server hosting
Reverse Proxy	Nginx	Load balancing, SSL
Load Balancing	Windows NLB / Nginx	Traffic distribution
CI/CD	Azure DevOps / Jenkins	Automated deployment
Secrets Management	Windows DPAPI / Vault	Credential storage

5.2 Integration Architecture

Integration Patterns for FMB:

1. **SAML 2.0 SSO** (Authentication)
 - o **Use Case:** Enterprise authentication via RSA Identity Provider
 - o **Protocol:** HTTPS/XML (SAML assertions)
 - o **Flows:** SP-initiated login, Single Logout (SLO)
 - o **Assertion Validation:** Signature verification, audience restriction, timestamp checks
2. **Batch ETL** (Data Integration)
 - o **Use Case:** Daily delta loads from Vault to App DB
 - o **Protocol:** SQL Server Linked Server / SSIS
 - o **Schedule:** Daily scheduled time (configurable)
 - o **Processing:** Staging → Validation → Production (MERGE UPSERT)

API Versioning Strategy: - **URL Versioning:** /api/v1/customers, /api/v2/customers - **Backward Compatibility:** Support current + previous version for Appropriate backward compatibility period - **Deprecation:** Adequate notice before version sunset

Integration Security: - **TLS 1.3:** All external communications encrypted - **IP Allowlisting:** Production APIs restricted to FMB network

5.3 API Catalog (Abridged)

Customer APIs: - GET /api/customers/search?q={query} - Intelligent customer search (SQL Server FTS) - GET /api/customers/:id - Get customer profile with nested data - GET /api/customers/:id/accounts - Get customer accounts - GET /api/customers/:id/transactions - Get customer transactions - GET /api/customers/:id/households - Get customer households - GET /api/customers/:id/notes - Get customer notes - POST /api/customers/:id/notes - Create customer note - PATCH /api/notes/:id - Update note (creates new version) - DELETE /api/notes/:id - Soft delete note

Account APIs: - GET /api/accounts/:id - Get account details - GET /api/accounts/:id/owners - Get account owners - GET /api/accounts/:id/transactions - Get account transactions - GET /api/accounts/:id/debit-cards - Get debit cards for account

Household APIs: - GET /api/households/:id - Get household details - GET /api/households/:id/members - Get household members - GET /api/households/:id/accounts - Get household accounts

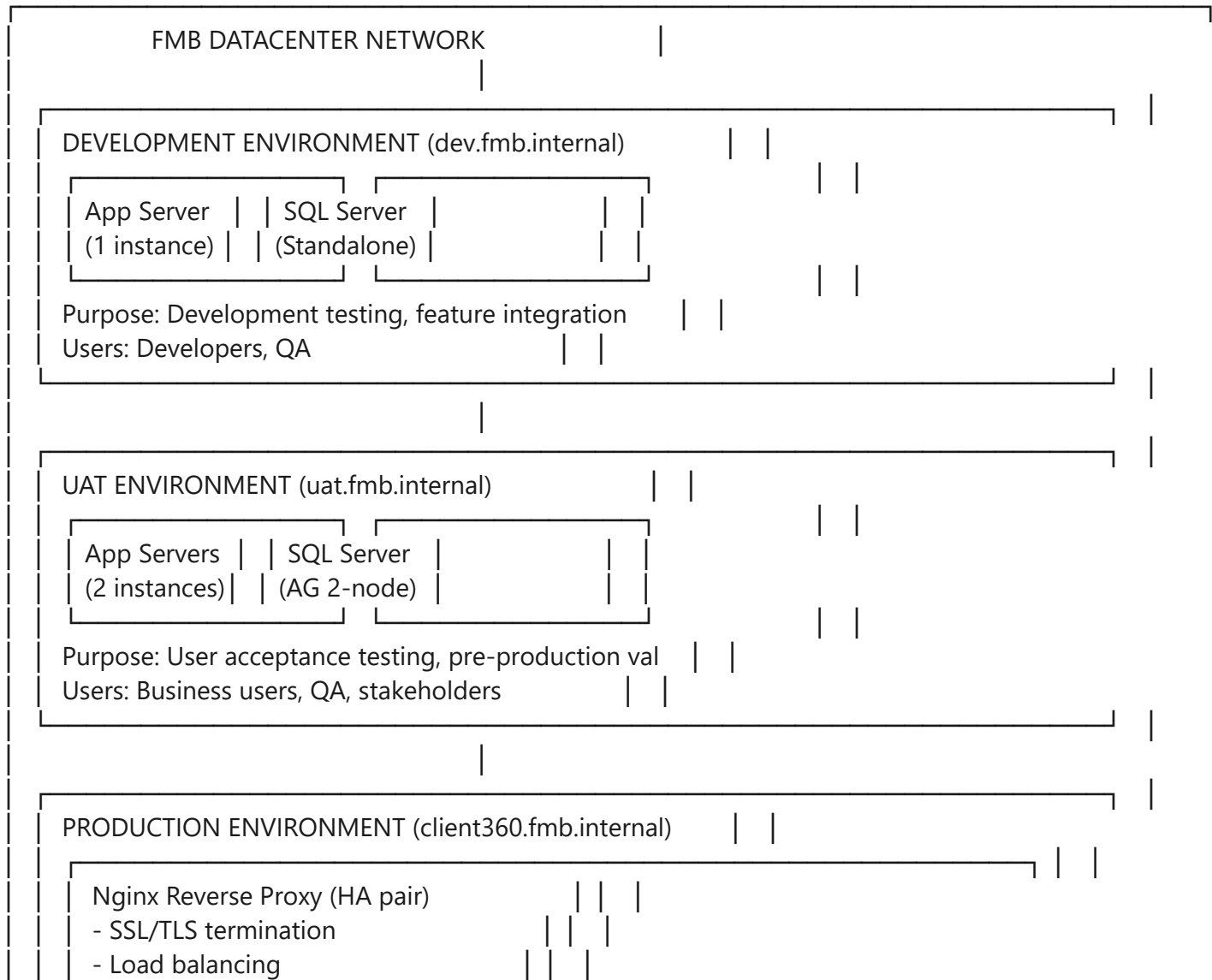
Authentication APIs: - GET /saml/login - Initiate SAML login (RSA IdP) - POST /saml/acs - Assertion Consumer Service - GET /saml/metadata - Service Provider metadata - GET /saml/logout - Initiate logout

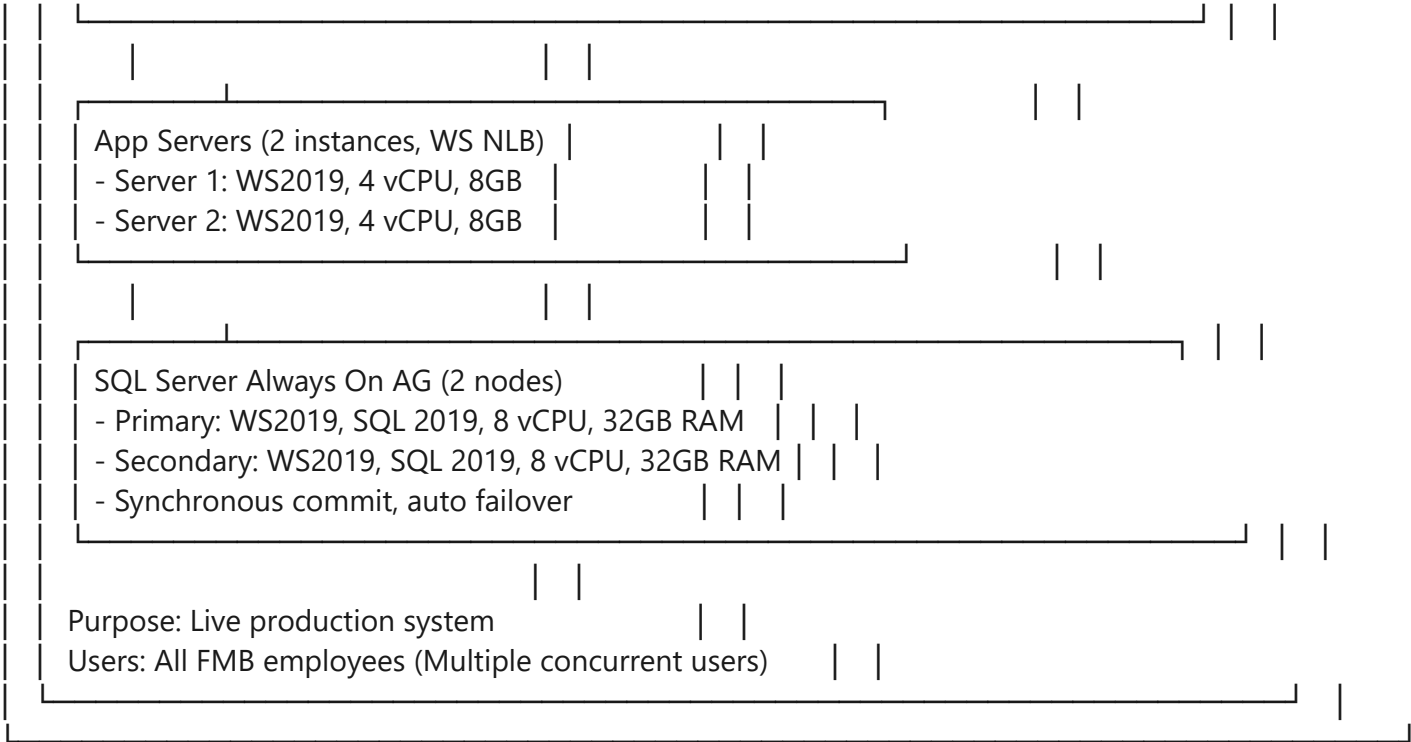
Authorization APIs: - GET /api/auth/permissions - Get user permissions - POST /api/auth/check-permission - Check conditional permission - GET /api/admin/users - List users (admin only) - POST /api/admin/users/:id/roles/manual - Assign role (admin only)

6. Infrastructure and Deployment Architecture

6.1 FMB Deployment Topology (Three Environments)

Environment Architecture:





Environment Specifications:

Environment	App Servers	SQL Server	Nginx	Purpose
Development	1 (4 vCPU, 8GB)	Standalone	No	Feature dev, integration testing
UAT	2 (4 vCPU, 8GB)	AG 2-node	Optional	User acceptance testing, pre-prod validation
Production	2 (4 vCPU, 8GB)	AG 2-node	HA pair	Live system, high availability SLA

6.2 Deployment Strategy for FMB

Deployment Process (Dev → UAT → Production):

- 1. Development Environment
 - o Developers commit code to develop branch
 - o Azure DevOps / Jenkins triggers build pipeline
 - o Automated tests run (unit, integration)

- o Deploy to dev environment for smoke testing

2. **UAT Environment**

- o Merge develop to uat branch after dev testing
- o Deploy to UAT environment
- o Business users perform acceptance testing
- o QA performs regression testing
- o Sign-off required from Product Owner

3. **Production Environment**

- o Merge uat to main branch after UAT sign-off
- o Create release tag (e.g., v1.2.0)
- o Production deployment window: Scheduled deployment window
- o Blue-green deployment strategy (zero-downtime)
- o Monitor metrics for 1 hour post-deployment

Deployment Steps (Production):

Step 1: Pre-deployment checks

- Verify SQL Server AG health (both nodes online)
- Verify Nginx reverse proxy health
- Backup current application version

Step 2: Database migrations (if any)

- Run SQL migration scripts on primary AG node
- Validate schema changes on secondary replica

Step 3: Deploy new application version

- Deploy to standby app server (blue)
- Start Node.js application on blue server
- Run smoke tests on blue server

Step 4: Traffic cutover

- Update Nginx config to route Limited traffic to blue
- Monitor error rates, response time for Short interval
- Gradually increase to gradual increase to full traffic

Step 5: Post-deployment validation

- Run automated smoke tests
- Verify health checks passing
- Monitor application metrics for 1 hour

Step 6: Decommission old version

- Keep green environment for Standard session duration (rollback window)
- After validation period, decommission green servers

6.3 CI/CD Pipeline for FMB (Azure DevOps / Jenkins)

Build Pipeline:

Azure DevOps Pipeline Example

trigger:

branches:

include:

- develop
- uat
- main

pool:

name: 'FMB-OnPrem-Agents' *# On-premise build agents*

stages:

- stage: Build

jobs:

- job: BuildAndTest

steps:

- task: NodeTool@0

inputs:

versionSpec: '20.x'

- script: npm install

displayName: 'Install dependencies'

- script: npm run lint

displayName: 'Run linter'

- script: npm run check

displayName: 'TypeScript type checking'

- script: npm run test

displayName: 'Run unit tests'

- script: npm run build

displayName: 'Build frontend'

- task: CopyFiles@2

inputs:

contents: '**'

targetFolder: '\$(Build.ArtifactStagingDirectory)'

```
- task: PublishBuildArtifacts@1
  inputs:
    pathToPublish: '$(Build.ArtifactStagingDirectory)'
    artifactName: 'client360-app'

- stage: DeployDev
  condition: eq(variables['Build.SourceBranch'], 'refs/heads/develop')
  jobs:
    - deployment: DeployToDev
      environment: 'Development'
      strategy:
        runOnce:
          deploy:
            steps:
              - script: |
                  xcopy /E /Y "$(Pipeline.Workspace)\client360-app\*" "\\dev-app01\c$\client360\"
                  sc.exe \\dev-app01 stop "Client360Service"
                  sc.exe \\dev-app01 start "Client360Service"
                displayName: 'Deploy to Dev Server'

- stage: DeployUAT
  condition: eq(variables['Build.SourceBranch'], 'refs/heads/uat')
  jobs:
    - deployment: DeployToUAT
      environment: 'UAT'
      strategy:
        runOnce:
          deploy:
            steps:
              - script: |
                  # Deploy to UAT servers (2 app servers)
                  xcopy /E /Y "$(Pipeline.Workspace)\client360-app\*" "\\uat-app01\c$\client360\"
                  xcopy /E /Y "$(Pipeline.Workspace)\client360-app\*" "\\uat-app02\c$\client360\"
                  # Restart services
                  sc.exe \\uat-app01 restart "Client360Service"
                  sc.exe \\uat-app02 restart "Client360Service"
                displayName: 'Deploy to UAT Servers'

- stage: DeployProduction
  condition: eq(variables['Build.SourceBranch'], 'refs/heads/main')
  jobs:
    - deployment: DeployToProduction
      environment: 'Production'
      strategy:
        runOnce:
          deploy:
            steps:
              - script: |
                  # Blue-green deployment (deploy to standby server first)
```

```
xcopy /E /Y "$(Pipeline.Workspace)\client360-app\*" "\\prod-app02\c$\client360\"
sc.exe \\prod-app02 restart "Client360Service"
# Wait for smoke tests
timeout /t 60
# Update Nginx config to route traffic
displayName: 'Deploy to Production (Blue-Green)'
```

Quality Gates: - All tests passing (unit, integration) - Strong code coverage - No critical security vulnerabilities (npm audit) - TypeScript compilation successful - SQL migrations tested on UAT

7. Cross-Cutting Technical Capabilities

7.1 Security Architecture for FMB

Authentication Flow (RSA SAML 2.0 SSO):

1. User accesses <https://client360.fmb.internal> → no session → redirect to </saml/login>
2. App generates SAML AuthnRequest → POST to RSA IdP
3. User authenticates at RSA IdP (username/password + MFA)
4. IdP generates SAML Response → POST to <https://client360.fmb.internal/saml/acs>
5. App validates SAML assertion:
 - Signature verification (X.509 certificate from RSA)
 - Audience restriction (must match FMB SP entity ID)
 - Timestamp validation (NotBefore, NotOnOrAfter)
 - Assertion expiry (Short timeframe)
6. App extracts attributes:
 - Employee ID (required)
 - Email, name, department (optional)
 - SAML role (e.g., 'System_Admin', 'Branch_Manager')
7. App looks up/creates employee record in SQL Server
8. App processes SAML role → auto-assign application role (RBAC)
9. App creates session in SQL Server session store (24-hour TTL)
10. App redirects to home page with secure session cookie

RSA SAML Configuration:

```
<!-- Service Provider Metadata (client360.fmb.internal) -->
<EntityDescriptor entityID="https://client360.fmb.internal">
  <SPSSODescriptor>
    <KeyDescriptor use="signing">
      <X509Certificate>MIIDXTCCAkWgAwIBAgIJAK...</X509Certificate>
    </KeyDescriptor>
    <AssertionConsumerService
      Binding="urn:oasis:names:tc:SAML:2.0:bindings:HTTP-POST"
      Location="https://client360.fmb.internal/saml/acs"
      index="1"/>
    </SPSSODescriptor>
  </EntityDescriptor>
```

Authorization Flow (RBAC + ABAC):

1. User makes API request → session cookie included
2. Authentication middleware validates session:
 - Check SQL Server session table (sessionId → employeeId)
 - Validate expiry (Standard session duration)
3. Authorization middleware checks permission:
 - Call permissionService.checkPermission(employeeId, permissionCode)
 - Evaluate RBAC: Does user's role grant this permission?
 - Evaluate ABAC: Do attributes match conditions?
Example: Branch restriction (employee.branchId = customer.branchId)
4. If denied:
 - Log denial to audit log (employee, permission, reason, resource)
 - Return HTTP 403 Forbidden
5. If granted:
 - Proceed to business logic
 - Log action to audit log (employee, action, resource, timestamp)

Data Protection:

Data State	Protection Mechanism	FMB Implementation
At Rest	SQL Server TDE	Database files encrypted (AES-256)
In Transit	TLS 1.3	Nginx reverse proxy, all HTTPS
In Use	PII masking	SSN: *--1234, account numbers never logged
In Logs	PII redaction	Custom logger redacts SSN, passwords

Audit Logging:

All user actions and security events are logged to SQL Server audit tables:

```
CREATE TABLE audit_log (  
  audit_id BIGINT IDENTITY(1,1) PRIMARY KEY,  
  event_timestamp DATETIME2 DEFAULT GETUTCDATE(),  
  event_type NVARCHAR(50), -- 'PERMISSION_DENIED', 'CUSTOMER_VIEW', 'NOTE_CREATED'  
  employee_id BIGINT,  
  employee_name NVARCHAR(200),  
  permission_code NVARCHAR(100),  
  resource_type NVARCHAR(50),  
  resource_id NVARCHAR(50),  
  denial_reason NVARCHAR(MAX),  
  ip_address NVARCHAR(50),
```

```
user_agent NVARCHAR(500),
session_id NVARCHAR(100)
);
```

-- Index for fast lookups

```
CREATE INDEX idx_audit_employee ON audit_log(employee_id, event_timestamp);
CREATE INDEX idx_audit_event_type ON audit_log(event_type, event_timestamp);
```

Log Retention: 7 years (regulatory requirement), indexed for search.

7.2 Monitoring and Observability

Application Metrics:

Key performance indicators monitored in production: - **Request Metrics:** HTTP request rate, error rate, latency (p50, p95, p99) - **Database Metrics:** Query execution time, connection pool usage, Always On AG status - **Session Metrics:** Active sessions, session creation/destruction rate - **Security Metrics:** Failed login attempts, permission denials, suspicious activity - **Business Metrics:** Customer searches per hour, notes created, households viewed

Health Checks:

```
// Health check endpoint
app.get('/health', async (req, res) => {
  const health = {
    status: 'healthy',
    timestamp: new Date().toISOString(),
    checks: {
      database: await checkDatabaseConnection(),
      sessions: await checkSessionStore(),
      api: 'ok'
    }
  };

  const isHealthy = Object.values(health.checks).every(c => c === 'ok');
  res.status(isHealthy ? 200 : 503).json(health);
});
```

Logging:

- **Application Logs:** Winston logger with structured JSON output
- **Audit Logs:** SQL Server audit tables for notes and role management (7-year retention)
- **System Logs:** Windows Event Log integration

7.3 Backup and Data Protection

Backup Strategy: - **SQL Server Backups:** Regular full, differential, and transaction log backups - **Retention:** Short-term online, mid-term nearline, long-term archival (compliance) - **Backup Validation:** Monthly restore test to dev environment

SQL Server Always On Availability Groups: - **Primary Node:** Read/write operations - **Secondary Node:** Asynchronous replication, read-only queries - **Automatic Failover:** Fast failover time - **Health Monitoring:** Automatic health detection and failover

Date: November 13, 2025

Document Owner: Enterprise Architecture & Engineering

Distribution: FMB Technology Leadership, Engineering Teams, Solution Architects

Executive Summary

The ClientIQ platform is an enterprise-grade customer intelligence system deployed on Farmers and Merchants Bank's **on-premise Windows Server infrastructure**. The system provides a unified 360-degree view of customer relationships, accounts, transactions, and household information, consolidating data from FMB's System of Record (SOR) through the Vault data warehouse into a high-performance application database.

Deployment Architecture: - **Infrastructure:** On-premise Windows Server 2019+ cluster - **Database:** Microsoft SQL Server 2019+ (Always On Availability Groups) - **Reverse Proxy:** Nginx for load balancing and SSL termination - **Authentication:** RSA SAML SSO integration - **Environments:** Development, UAT (User Acceptance Testing), Production - **Data Pipeline:** SOR → Vault → Application Database (initial bulk load + daily delta updates)

Key Technology Stack: - Frontend: React 18 + TypeScript + Material-UI - Backend: Express.js + Node.js hosted on Windows Server - Database: Microsoft SQL Server with Full-Text Search - Session Management: SQL Server session store - Authentication: RSA Identity Provider via SAML 2.0

1. Introduction

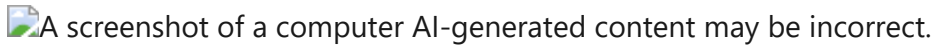
1.1 System Overview

The ClientIQ platform consolidates customer data from FMB's System of Record through the Vault data warehouse, delivering a comprehensive 360-degree view of customer relationships, accounts, transactions, and household information. The system is designed specifically for FMB's on-premise Windows Server infrastructure, leveraging Microsoft SQL Server as the primary data store.

Core Capabilities: - **Customer Intelligence:** Unified customer profiles with household relationships, account aggregation, and transaction history - **Customer Data Management:** Customer information viewing with support for individual, business, trust, and estate types - **Account Management:** Comprehensive account viewing with multi-owner support, debit card information, and transaction history - **Search & Discovery:** Intelligent fuzzy search powered by SQL Server Full-Text Search with relevance ranking - **Customer Notes:** Full CRUD operations on customer notes with versioning and audit trails

Technical Foundation: - React 18 + TypeScript single-page application - Express.js REST API layer hosted on Node.js (Windows Server) - Microsoft SQL Server 2019+ with Always On Availability Groups - RSA SAML 2.0 SSO authentication - Nginx reverse proxy for SSL termination and load balancing

1.2 High Level Architecture Overview

A screenshot of a computer AI-generated content may be incorrect.

Note: The diagram illustrates key Client IQ components, though it's not exhaustive.

The solution is to be built as a responsive web application that will extend to mobile platforms in the future. It supports Single Sign-On (SSO) for seamless and secure access.

The application architecture is organized into multiple logical layers:

Interface Layer: Web user interface supporting responsive design and secure client interactions.

Application Services Layer: Implements business logic for client data presentation, entitlement enforcement, and user management.

Integration Layer: Provides secure API and service connectivity with JHA core banking and supporting systems.

Data Layer: Consumes curated data from Vault and SPOT sources and supports data engineering functions for performance, enrichment, and governance.

The architecture includes key foundational components to ensure security, reliability, and operational excellence:

Security: SSO integration, TLS encryption, and user entitlements using RBAC and ABAC models.

Observability: Centralized logging, application health checks, and alerting for proactive monitoring.

DevSecOps Enablement: Automated CI/CD pipelines, code scanning (SAST/DAST), and controlled environment promotion for safe and efficient delivery.

1.3 Technical Architecture

Layered Architecture Pattern: The system employs a clear separation of concerns across five layers:

1. **Presentation Layer:** React components with Material-UI, responsive design

2. **API Layer:** RESTful endpoints with versioning
3. **Business Logic Layer:** Service adapters with PII masking, permission checks
4. **Data Access Layer:** SQL Server parameterized queries, stored procedures, connection pooling
5. **Integration Layer:** ETL pipelines (SOR/Vault), SAML authentication

Architectural Patterns: - **DTO Pattern:** Data Transfer Objects with PII masking via customer adapters - **ETL Pipeline:** Scheduled data loads from SOR → Vault → App DB (initial bulk + daily deltas) - **Repository Pattern:** Data access abstraction through SQL Server stored procedures - **Session Management:** SQL Server session store with secure cookie configuration

Technology Strategy: - **Windows Integration:** Node.js hosted on Windows Server with Nginx reverse proxy - **SQL Server Optimization:** Full-Text Search indexes, query optimization, Always On AG - **Scalability:** Horizontal scaling via Windows Network Load Balancing - **Developer Experience:** TypeScript-first development, automated testing, CI/CD

1.4 Security Posture

The platform implements defense-in-depth security controls following banking industry best practices:

Authentication & Authorization: - **Enterprise SSO:** SAML 2.0 integration with RSA Identity Provider - **Session Management:** Secure cookies (HttpOnly, SameSite, Secure), SQL Server session store - **Access Control:** RBAC (Multiple roles and privilege levels) + ABAC (attribute-based permissions) - **Permission Enforcement:** Every API request validated against user permissions

Data Protection: - **In Transit:** TLS 1.3 encryption via Nginx reverse proxy - **At Rest:** SQL Server Transparent Data Encryption (TDE) - **In Use:** PII masking (SSN: --1234, Tax ID: --5678) - **In Logs:** Sensitive data redacted (no SSN, account numbers, passwords)

Network Security: - Nginx reverse proxy for load balancing and SSL termination - Firewall rules restricting access to FMB corporate network - IDS/IPS integration for threat detection

Audit Logging: - Audit trails for customer notes and role management - 7-year retention in SQL Server audit tables

1.5 Architectural Principles

The platform is architected according to enterprise-grade principles ensuring scalability, security, and maintainability:

Principle	Implementation
On-Premise First	Windows Server cluster, SQL Server Always On, on-prem Nginx
Security by Design	Defense-in-depth: TLS 1.3, TDE encryption, RBAC/ABAC, RSA SAML SSO, PII masking
Enterprise Integration	SOR → Vault → App DB pipeline, SSIS ETL workflows
API First	RESTful API-driven architecture with versioning
Data Sovereignty	Role-based permissions, data masking
Regulatory Compliance	Designed to support compliance requirements
Windows Native	Node.js on Windows, SQL Server integration, Active Directory compatibility
High Availability	SQL Server Always On Availability Groups, automatic failover

1.6 Key Non-Functional Requirements

Performance: - Transaction response time: fast response time - Search query response: fast search performance via SQL Server Full-Text Search - Customer profile load: quick loading - Support multiple concurrent FMB staff users - Handle large customer databases with fast search capabilities

Scalability: - Horizontal scaling of application tier via Windows Network Load Balancing - SQL Server read replicas for query distribution - SQL Server session store for stateful session management - Support for enterprise-scale customer databases

Availability: - high availability SLA - SQL Server Always On Availability Groups for automatic failover - Nginx reverse proxy with health checks and failover

Security: - End-to-end TLS 1.3 encryption - Data masking for PII in UI and logs - Audit logging for notes and role management with 7-year retention

Compliance Standards: - Customer data protected with encryption (TLS 1.3, SQL Server TDE) - Audit logging for notes and role management with 7-year retention - PII masking in UI and logs - Role-based access control for data segregation

2. System Context

2.1 System Context Diagram

The ClientIQ system operates within FMB’s on-premise datacenter environment:

FMB BANK EMPLOYEES

- Tellers (Branch Operations)
- Relationship Managers (Customer Service)
- Branch Managers (Team Oversight)
- Executives (Analytics & Reporting)
- System Administrators (IT Operations)

HTTPS (Port 443)

Corporate Network / VPN

FMB DATACENTER - PERIMETER SECURITY

Firewall / IDS/IPS

- Port 443 (HTTPS only)
- Source IP whitelisting
- DDoS protection

Nginx Reverse Proxy (Load Balancer)

- SSL/TLS termination (TLS 1.3)
- Load balancing (round-robin, sticky sessions)
- Health checks (HTTP 200 on /health endpoint)

App Server 1
(Windows
Server 2019)

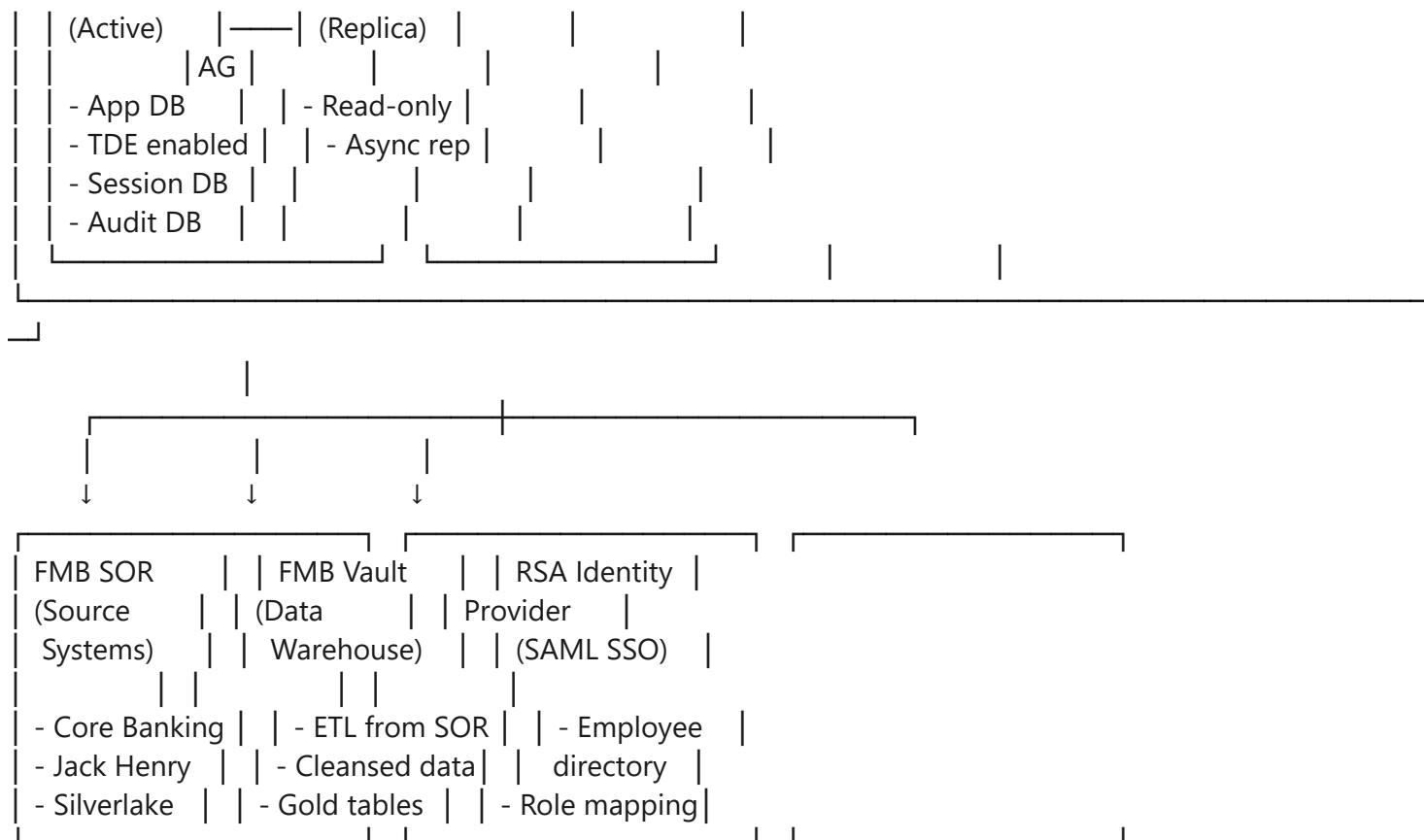
App Server 2
(Windows
Server 2019)

- Node.js
- Express.js
- React SPA

- Node.js
- Express.js
- React SPA

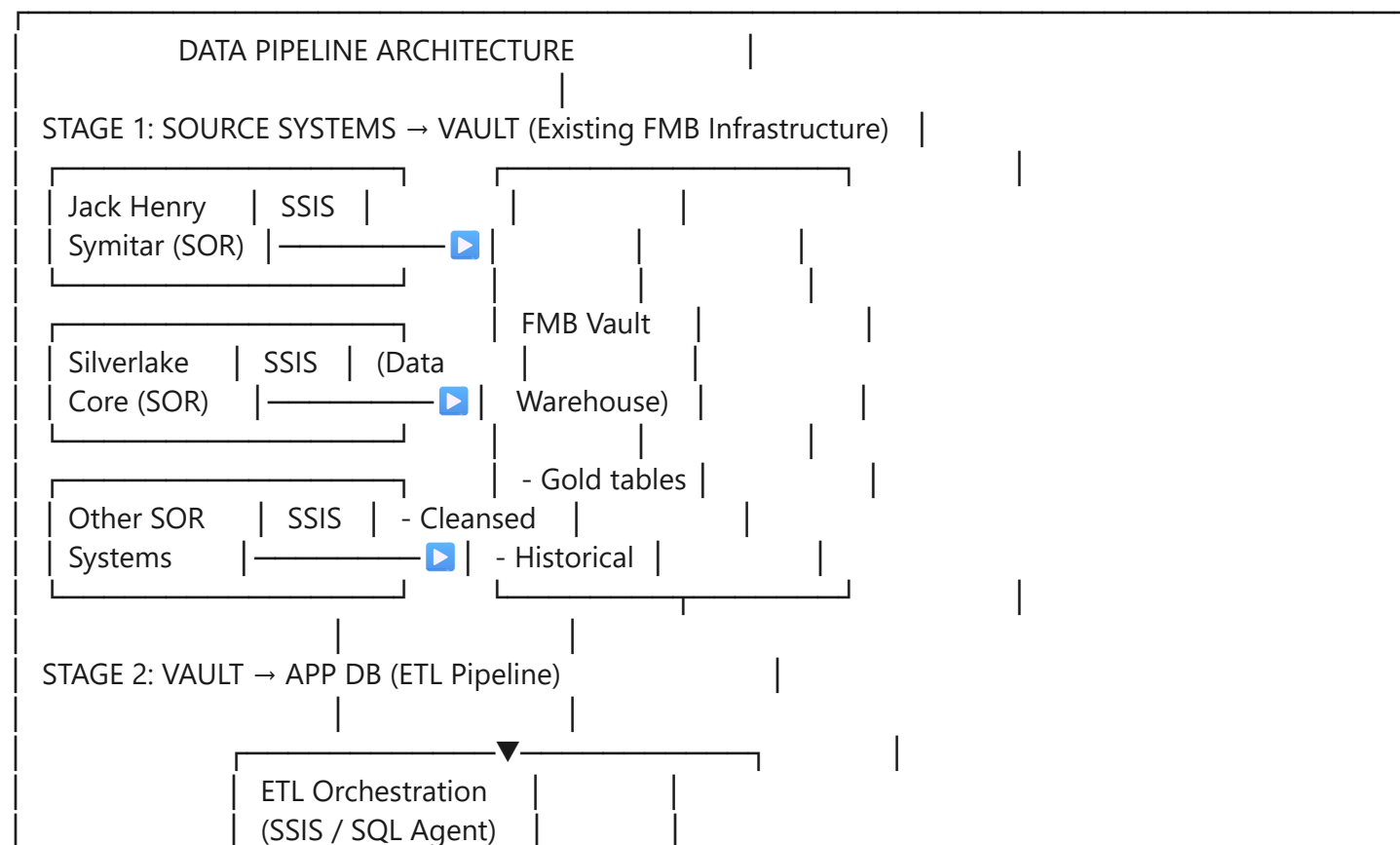
SQL Server
Primary

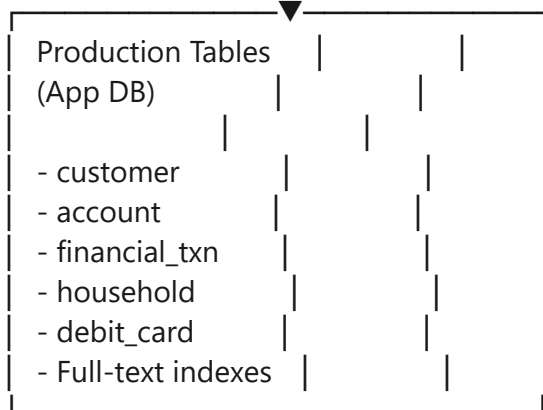
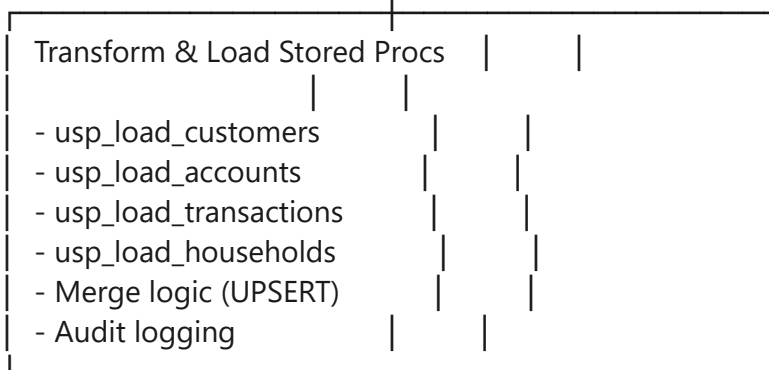
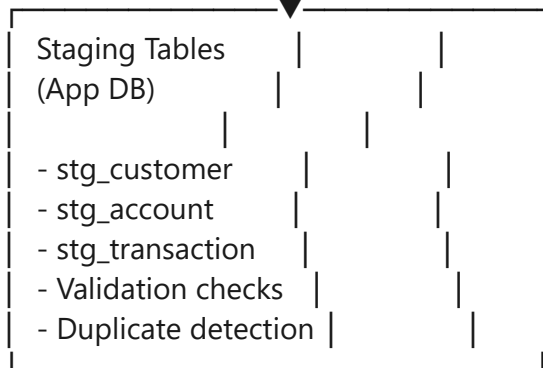
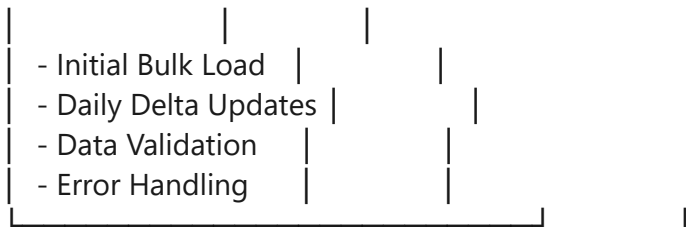
SQL Server
Secondary



2.2 FMB Data Pipeline Architecture (SOR → Vault → App DB)

The application database is populated through a multi-stage ETL pipeline:





SCHEDULE:

- Initial Load: One-time bulk import (all historical data)
- Delta Loads: Daily during scheduled maintenance window (incremental updates)

Data Load Strategy:

Load Type	Frequency	Volume	Method	Window
Initial Bulk Load	One-time	Large volume of customers and accounts	Full table truncate/reload	Weekend maintenance window
Daily Delta Load	Daily scheduled time	Changed records	Incremental MERGE (UPSERT)	Short maintenance window
Error Handling	Real-time	N/A	Staging table validation, error logging	Alert on failure
Data Validation	Every load	All records	Referential integrity, business rules	Rollback on critical errors

2.3 External System Interfaces

System	Integration Type	Protocol	Data Flow	Authentication
FMB SOR (Jack Henry)	ETL Batch	ODBC/SSIS	Inbound (SOR → Vault)	Windows Auth
FMB SOR (Silverlake)	ETL Batch	ODBC/SSIS	Inbound (SOR → Vault)	Windows Auth
FMB Vault	ETL Batch	SQL Server Link	Inbound (Vault → App DB)	Windows Auth
RSA Identity Provider	SAML 2.0	HTTPS/XML	Bidirectional (auth)	X.509 Certificate

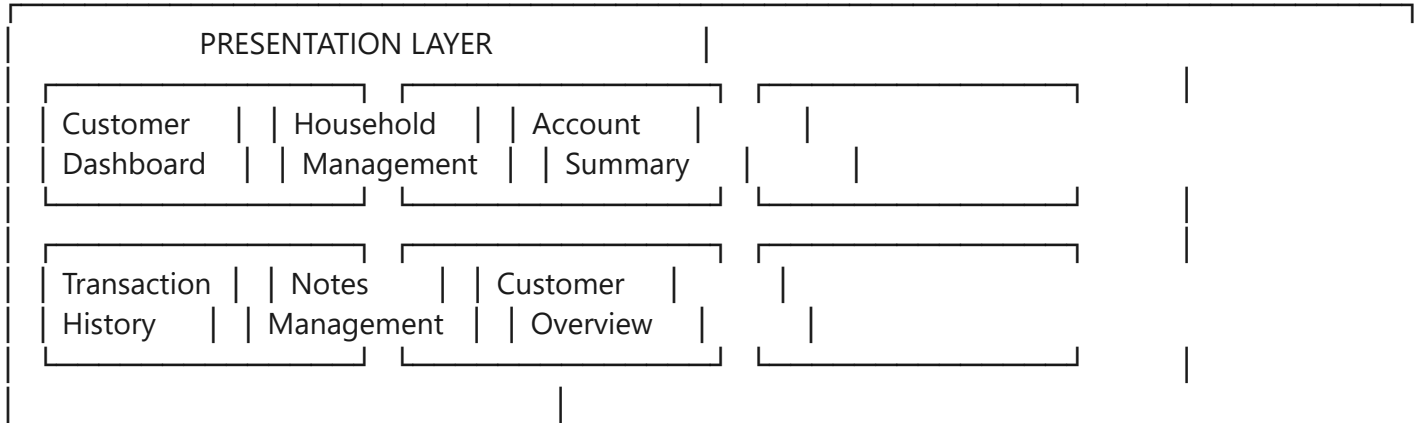
2.4 Stakeholder Analysis

Stakeholder Group	Primary Needs	System Interaction	Success Metrics
Tellers	Fast customer lookup, transaction viewing	Read-only customer profiles, account balances	Fast customer lookup
Relationship Managers	Comprehensive customer intelligence, notes	Full CRUD on customer notes, household management	360° view completeness
Branch Managers	Team oversight, customer service management	All RM capabilities + team reports	High system usage, customer satisfaction
Executives	Strategic analytics, portfolio insights	Dashboards, executive reports (read-only)	Strategic insights availability
System Administrators	User management, system configuration	Full administrative access, role assignment	High system availability
IT Operations	Monitoring, incident response	Infrastructure management, log analysis	MTTR Quick

3. Logical Component Architecture

3.1 Component Structure and Responsibilities

The platform follows a **layered architecture pattern** with clear separation of concerns:



Technology: React 18, TypeScript, Material-UI, shadcn/ui
State Management: TanStack React Query (server state)
Routing: Wouter (SPA routing)
Forms: React Hook Form + Zod validation

↕↕ HTTPS/JSON

API LAYER

API Gateway (Express.js on Windows Server)
- Route definitions (/api/customers, /api/accounts...)
- Request validation (Zod schemas)
- Error handling (standardized error responses)

Authentication Middleware
- SAML 2.0 SSO (npm saml + RSA IdP)
- Session validation (SQL Server session store)
- Employee lookup/upsert from SAML assertions

Authorization Middleware
- RBAC permission checks (role-based)
- ABAC conditional evaluation (attribute-based)
- Privilege level verification

↕↕

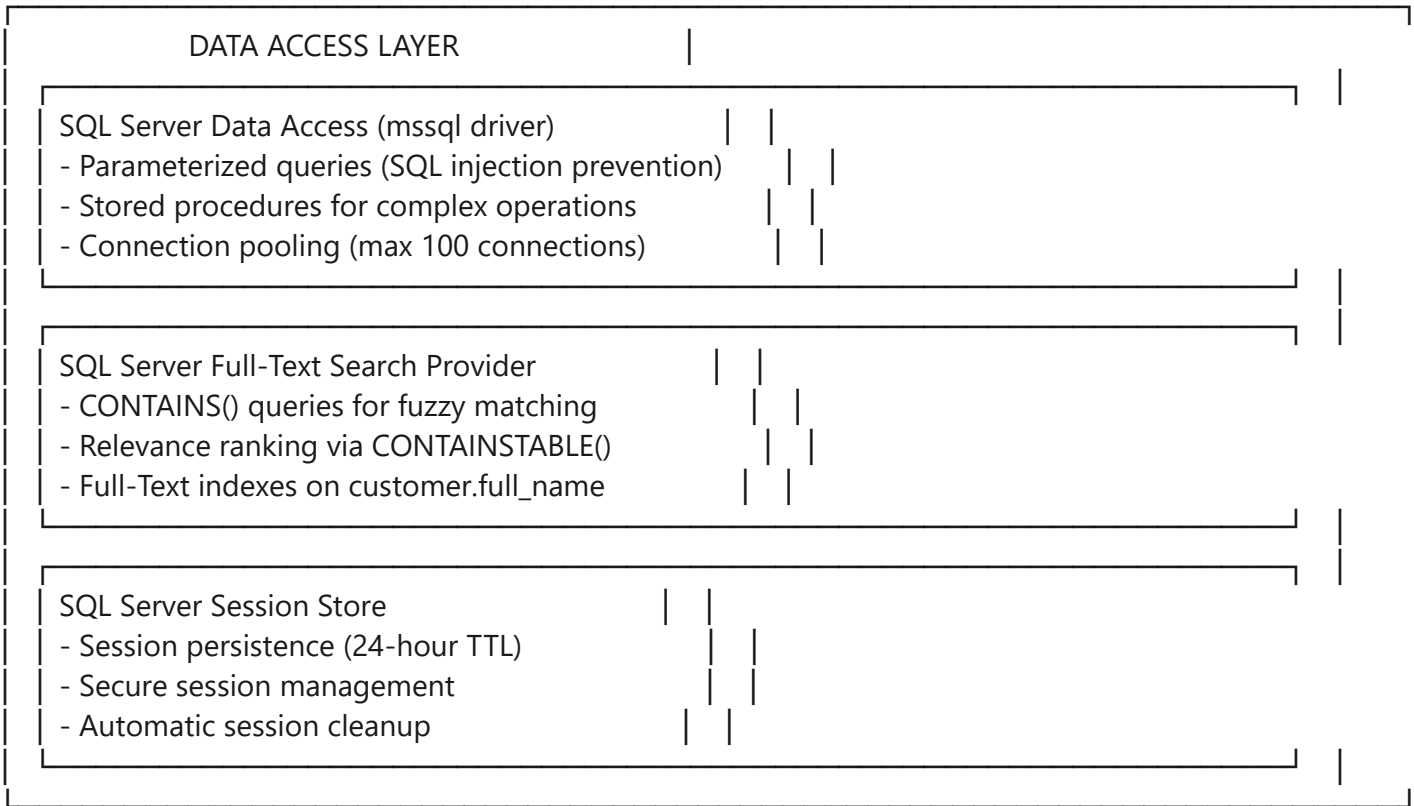
BUSINESS LOGIC LAYER

Customer Adapter	Account Service	Household Service
- PII masking	- Balance calc	- Aggregation
- DTO mapping	- Owner lookup	- Members

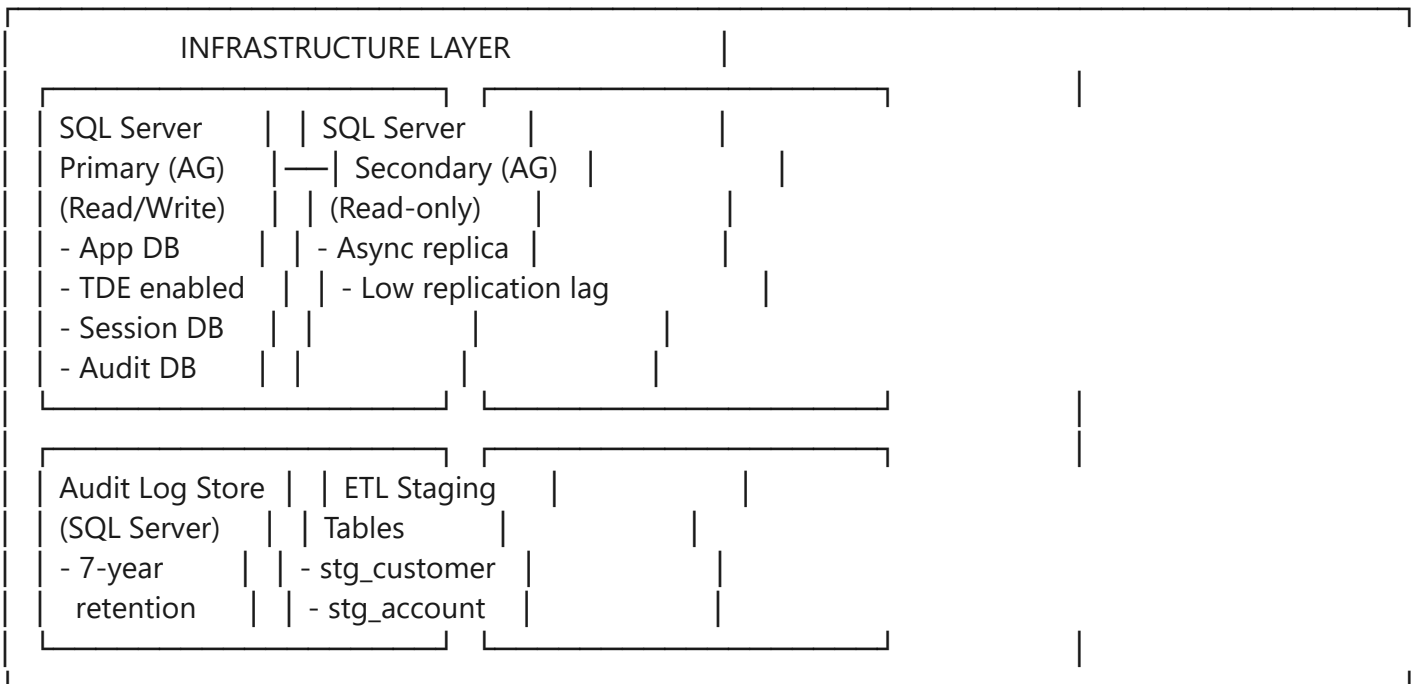
Transaction Service	Search Service	Data Adapter
- Category map	- FTS query	- Field map
- Balance run	- Relevance	- Validation

Permission Service	Note Service
- ABAC eval	- Versioning
- Audit log	- Soft delete

↓↑



↓↑



4. Data Architecture and Information Management

4.1 Data Storage Strategy

SQL Server Primary Database: - **Primary Database:** Microsoft SQL Server 2019+ with Always On Availability Groups - **Read Replicas:** Secondary AG replica for query distribution (read-only) - **Session Store:** SQL Server

session table for stateful session management - **Audit Store:** SQL Server audit tables for 7-year regulatory retention - **Staging Tables:** ETL staging area for data validation before production load

Data Flow Patterns: 1. **Write Operations:** All writes to SQL Server primary (AG) 2. **Read Operations:** Distributed across primary and read replica 3. **Session Management:** SQL Server session store with automatic cleanup 4. **Audit Logs:** Write-through to audit tables (all user actions) 5. **Search Queries:** Optimized via SQL Server Full-Text Search indexes 6. **ETL Loads:** Vault → Staging → Production tables (daily 2:00 AM)

4.2 SQL Server Full-Text Search Configuration

Full-Text Catalog and Index:

```
-- Create Full-Text Catalog
CREATE FULLTEXT CATALOG ftCatalog_CustomerSearch AS DEFAULT;

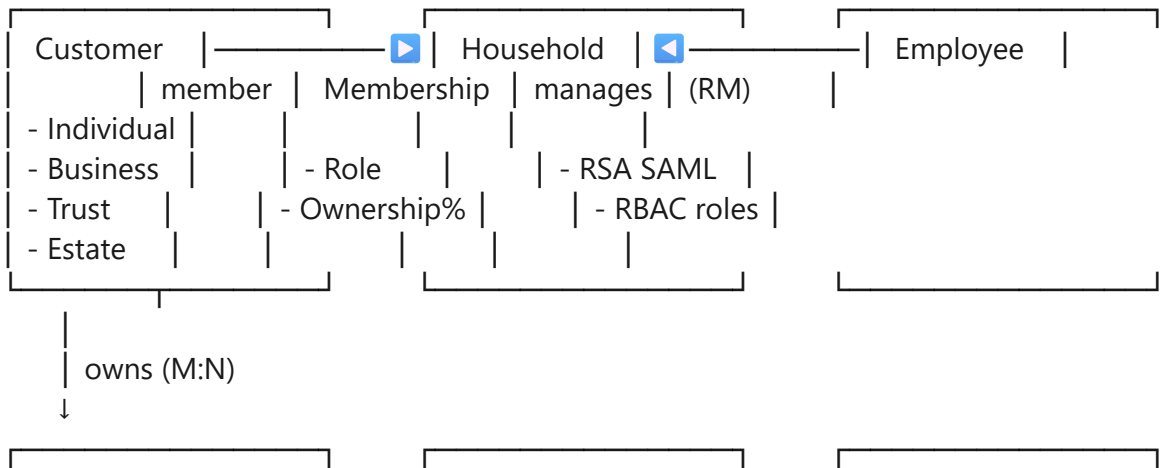
-- Create Full-Text Index on customer table
CREATE FULLTEXT INDEX ON customer(full_name)
KEY INDEX PK_customer_customer_id
ON ftCatalog_CustomerSearch
WITH CHANGE_TRACKING AUTO;

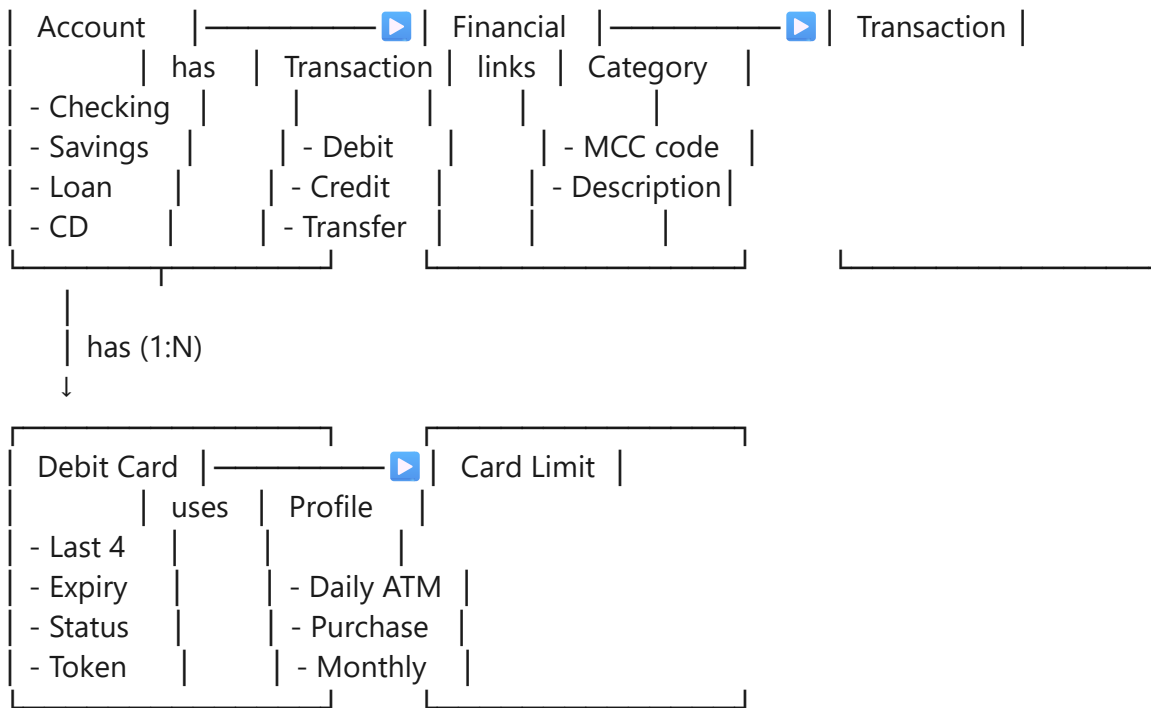
-- Search query example (CONTAINS)
SELECT customer_id, full_name, customer_type
FROM customer
WHERE CONTAINS(full_name, '"John*' OR "Smith*")
ORDER BY customer_id;

-- Search query with relevance ranking (CONTAINSTABLE)
SELECT c.customer_id, c.full_name, c.customer_type, ft.RANK
FROM customer c
INNER JOIN CONTAINSTABLE(customer, full_name, '"John Smith"') AS ft
  ON c.customer_id = ft.[KEY]
ORDER BY ft.RANK DESC;
```

4.3 Core Data Models

Entity Relationship Overview:





4.4 Database Schema Highlights (SQL Server)

Customer Table:

```

CREATE TABLE customer (
    customer_id BIGINT IDENTITY(1,1) PRIMARY KEY,

    -- Individual customer fields
    first_name NVARCHAR(100),
    last_name NVARCHAR(100),
    middle_name NVARCHAR(100),
    date_of_birth DATE,

    -- Business customer fields
    business_name NVARCHAR(200),
    dba_name NVARCHAR(200),

    -- Unified search field (always populated)
    full_name NVARCHAR(200) NOT NULL,

    -- Identification (masked in UI)
    tax_identifier NVARCHAR(20) UNIQUE,
    government_id NVARCHAR(50),

    -- Classification
    customer_type NVARCHAR(20) NOT NULL DEFAULT 'individual',
    customer_status NVARCHAR(20) DEFAULT 'active',

    -- Compliance
    kyc_status NVARCHAR(20),
  
```

```

risk_rating NVARCHAR(20),

-- Flags
is_employee BIT DEFAULT 0,
vip_customer BIT DEFAULT 0,
is_deceased BIT DEFAULT 0,

-- Core banking integration
jack_henry_cif_number NVARCHAR(20),
silverlake_customer_id NVARCHAR(20),

-- Audit
created_at DATETIME2 DEFAULT GETUTCDATE(),
updated_at DATETIME2 DEFAULT GETUTCDATE(),

-- Constraint: conditional names based on customer type
CONSTRAINT CK_customer_name_type CHECK (
    (customer_type = 'individual' AND first_name IS NOT NULL AND last_name IS NOT NULL) OR
    (customer_type IN ('business', 'trust', 'estate') AND business_name IS NOT NULL)
)
);

-- Full-Text Index for search
CREATE FULLTEXT INDEX ON customer(full_name)
KEY INDEX PK_customer_customer_id
WITH CHANGE_TRACKING AUTO;

-- Indexes for performance
CREATE INDEX idx_customer_tax_id ON customer(tax_identifier);
CREATE INDEX idx_customer_status ON customer(customer_status);
CREATE INDEX idx_customer_type ON customer(customer_type);

```

SQL Server Session Store:

```

CREATE TABLE sessions (
    session_id NVARCHAR(255) PRIMARY KEY,
    session_data NVARCHAR(MAX),
    expires_at DATETIME2,
    created_at DATETIME2 DEFAULT GETUTCDATE()
);

-- Index for session cleanup
CREATE INDEX idx_sessions_expires ON sessions(expires_at);

-- Automatic cleanup stored procedure
CREATE PROCEDURE usp_cleanup_expired_sessions
AS
BEGIN
    DELETE FROM sessions

```

```
WHERE expires_at < GETUTCDATE();  
END;  
GO
```

4.5 ETL Staging Tables (Vault → App DB)

Staging Table Example:

```
-- Staging table for customer data from Vault  
CREATE TABLE stg_customer (  
    load_id INT, -- ETL batch identifier  
    source_system NVARCHAR(50), -- 'JackHenry' or 'Silverlake'  
    customer_id_src NVARCHAR(50), -- Source system customer ID  
  
    -- Customer data fields (mirroring production table)  
    first_name NVARCHAR(100),  
    last_name NVARCHAR(100),  
    business_name NVARCHAR(200),  
    full_name NVARCHAR(200),  
    tax_identifier NVARCHAR(20),  
    customer_type NVARCHAR(20),  
  
    -- Data quality flags  
    is_valid BIT DEFAULT 1,  
    validation_errors NVARCHAR(MAX),  
  
    -- Audit  
    loaded_at DATETIME2 DEFAULT GETUTCDATE()  
);  
  
-- Stored procedure: Load customers from staging to production  
CREATE PROCEDURE usp_load_customers  
    @load_id INT  
AS  
BEGIN  
    SET NOCOUNT ON;  
  
    -- Validate staging data  
    UPDATE stg_customer  
    SET is_valid = 0,  
        validation_errors = 'Missing required fields'  
    WHERE load_id = @load_id  
        AND (full_name IS NULL OR customer_type IS NULL);  
  
    -- MERGE (UPSERT) into production table  
    MERGE INTO customer AS target  
    USING (  
        SELECT * FROM stg_customer  
        WHERE load_id = @load_id AND is_valid = 1
```

```
) AS source
ON target.jack_henry_cif_number = source.customer_id_src
   OR target.silverlake_customer_id = source.customer_id_src

WHEN MATCHED THEN
  UPDATE SET
    first_name = source.first_name,
    last_name = source.last_name,
    full_name = source.full_name,
    updated_at = GETUTCDATE()

WHEN NOT MATCHED THEN
  INSERT (first_name, last_name, business_name, full_name,
          tax_identifier, customer_type, jack_henry_cif_number)
  VALUES (source.first_name, source.last_name, source.business_name,
          source.full_name, source.tax_identifier, source.customer_type,
          source.customer_id_src);

-- Log results
DECLARE @inserted INT = @@ROWCOUNT;
PRINT 'Customers loaded: ' + CAST(@inserted AS NVARCHAR);

END;
GO
```

4.6 Data Retention and Privacy

Data Type	Retention Period	Deletion Method	Regulatory Basis
Customer PII	7 years post-account closure	Hard delete after retention	GLBA, state banking laws
Account Data	7 years post-closure	Archival then purge	Reg CC, Reg E, UCC
Transactions	7 years	Archival then purge	Regulatory requirements
Debit Card Data	7 years post-cancellation	Immediate tokenization, purge after retention	PCI DSS
Audit Logs	7 years minimum	Indexed storage in SQL Server	Regulatory requirements
Customer Notes	7 years or indefinite (legal hold)	Soft delete (is_deleted flag)	Internal policy
Session Data	Standard session duration	Automatic cleanup via stored procedure	Security policy
SAML Assertions	Regular interval	Rolling deletion	SAML spec

PII Handling: - **At Rest:** SQL Server TDE (Transparent Data Encryption) for database files - **In Transit:** TLS 1.3 for all network communication (Nginx reverse proxy) - **In Use:** PII fields masked in UI (SSN: --1234, Tax ID: --5678) - **In Logs:** PII fields redacted (never log SSN, account numbers, passwords)

5. Technology Stack and Integration Patterns

5.1 Technology Stack Specification for FMB

Frontend Technologies:

Category	Technology	Version	Purpose
UI Framework	React	18.3.1	Component-based UI
Language	TypeScript	5.6.3	Type-safe development
Build Tool	Vite	5.4.19	Fast dev server
UI Library	Material-UI (MUI)	7.3.2	Enterprise components
UI Components	shadcn/ui (Radix)	Latest	Accessible primitives
Styling	Tailwind CSS	3.4.17	Utility-first CSS
State Management	TanStack React Query	5.60.5	Server state caching
Routing	Wouter	3.3.5	SPA routing
Form Handling	React Hook Form	7.55.0	Performant forms
Validation	Zod	3.24.2	Schema validation
Date Handling	date-fns	3.6.0	Date manipulation
Charts	Recharts	2.15.2	Data visualization

Backend Technologies:

Category	Technology	Version	Purpose
Runtime	Node.js	20.x (LTS)	JavaScript runtime
Framework	Express.js	4.21.2	Web framework
Language	TypeScript	5.6.3	Type-safe backend
Database Driver	mssql	12.0.0	SQL Server driver
SAML Authentication	npm saml	Latest	SAML 2.0 SSO
Session Store	connect-mssql-v2	Latest	SQL Server sessions
Password Hashing	bcryptjs	3.0.2	Secure password storage
Validation	Zod	3.24.2	Request validation
Process Manager	tsx	4.19.1	TypeScript execution

Database Technologies:

Category	Technology	Version	Purpose
Primary Database	SQL Server	2019+	Relational database
Search	Full-Text Search	Built-in	Fuzzy text matching
High Availability	Always On AG	Built-in	Automatic failover
Encryption	TDE	Built-in	Data at rest encryption
Session Management	SQL Server Table	Built-in	Session persistence
ETL	SSIS / SQL Agent	2019+	Data pipeline

Infrastructure Technologies:

Category	Technology	Purpose
Operating System	Windows Server 2019+	App server hosting
Reverse Proxy	Nginx	Load balancing, SSL
Load Balancing	Windows NLB / Nginx	Traffic distribution
CI/CD	Azure DevOps / Jenkins	Automated deployment
Secrets Management	Windows DPAPI / Vault	Credential storage

5.2 Integration Architecture

Integration Patterns for FMB:

1. **SAML 2.0 SSO** (Authentication)
 - o **Use Case:** Enterprise authentication via RSA Identity Provider
 - o **Protocol:** HTTPS/XML (SAML assertions)
 - o **Flows:** SP-initiated login, Single Logout (SLO)
 - o **Assertion Validation:** Signature verification, audience restriction, timestamp checks
2. **Batch ETL** (Data Integration)
 - o **Use Case:** Daily delta loads from Vault to App DB
 - o **Protocol:** SQL Server Linked Server / SSIS
 - o **Schedule:** Daily scheduled time (configurable)
 - o **Processing:** Staging → Validation → Production (MERGE UPSERT)

API Versioning Strategy: - **URL Versioning:** /api/v1/customers, /api/v2/customers - **Backward Compatibility:** Support current + previous version for Appropriate backward compatibility period - **Deprecation:** Adequate notice before version sunset

Integration Security: - **TLS 1.3:** All external communications encrypted - **IP Allowlisting:** Production APIs restricted to FMB network

5.3 API Catalog (Abridged)

Customer APIs: - GET /api/customers/search?q={query} - Intelligent customer search (SQL Server FTS) - GET /api/customers/:id - Get customer profile with nested data - GET /api/customers/:id/accounts - Get customer accounts - GET /api/customers/:id/transactions - Get customer transactions - GET /api/customers/:id/households - Get customer households - GET /api/customers/:id/notes - Get customer notes - POST /api/customers/:id/notes - Create customer note - PATCH /api/notes/:id - Update note (creates new version) - DELETE /api/notes/:id - Soft delete note

Account APIs: - GET /api/accounts/:id - Get account details - GET /api/accounts/:id/owners - Get account owners - GET /api/accounts/:id/transactions - Get account transactions - GET /api/accounts/:id/debit-cards - Get debit cards for account

Household APIs: - GET /api/households/:id - Get household details - GET /api/households/:id/members - Get household members - GET /api/households/:id/accounts - Get household accounts

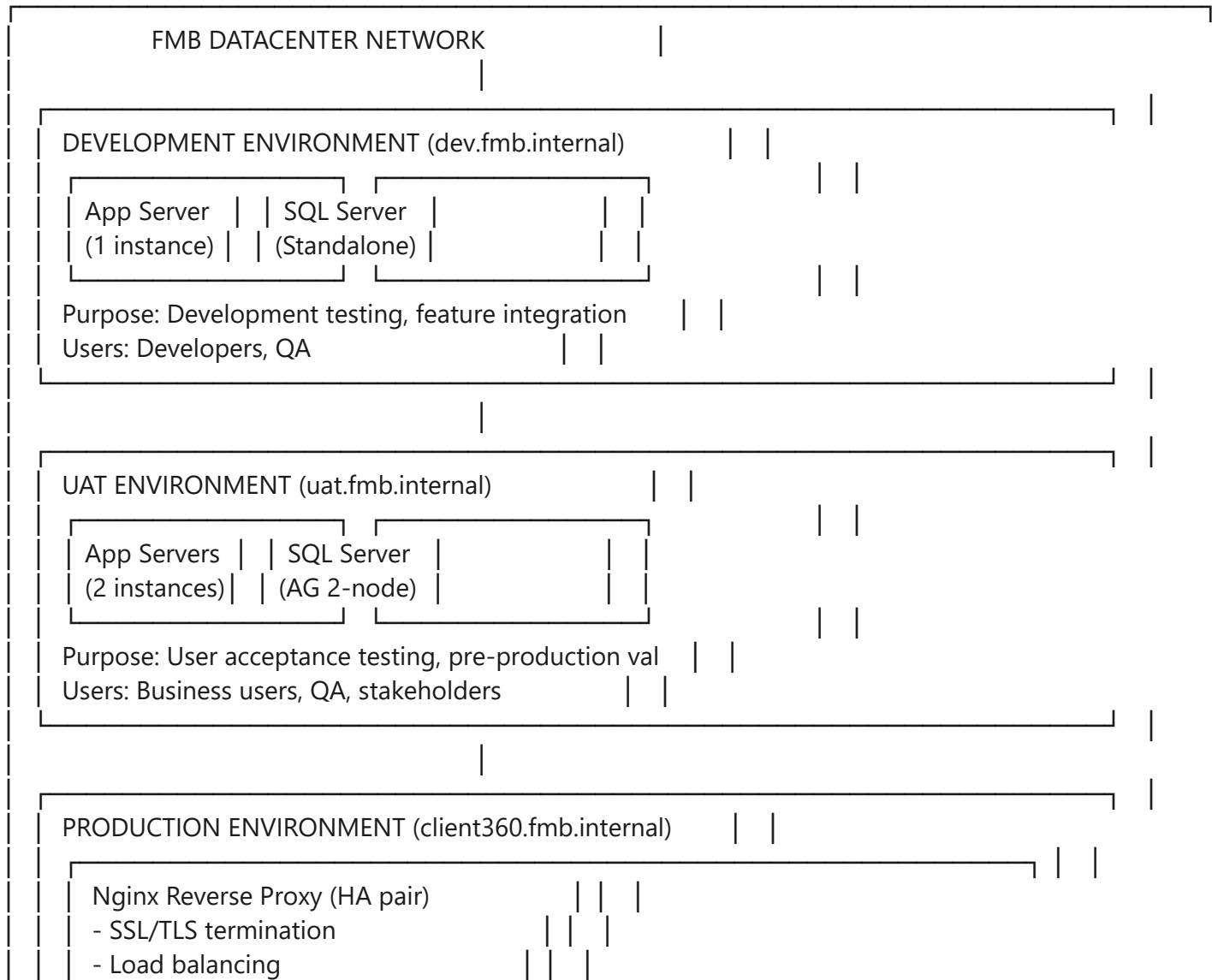
Authentication APIs: - GET /saml/login - Initiate SAML login (RSA IdP) - POST /saml/acs - Assertion Consumer Service - GET /saml/metadata - Service Provider metadata - GET /saml/logout - Initiate logout

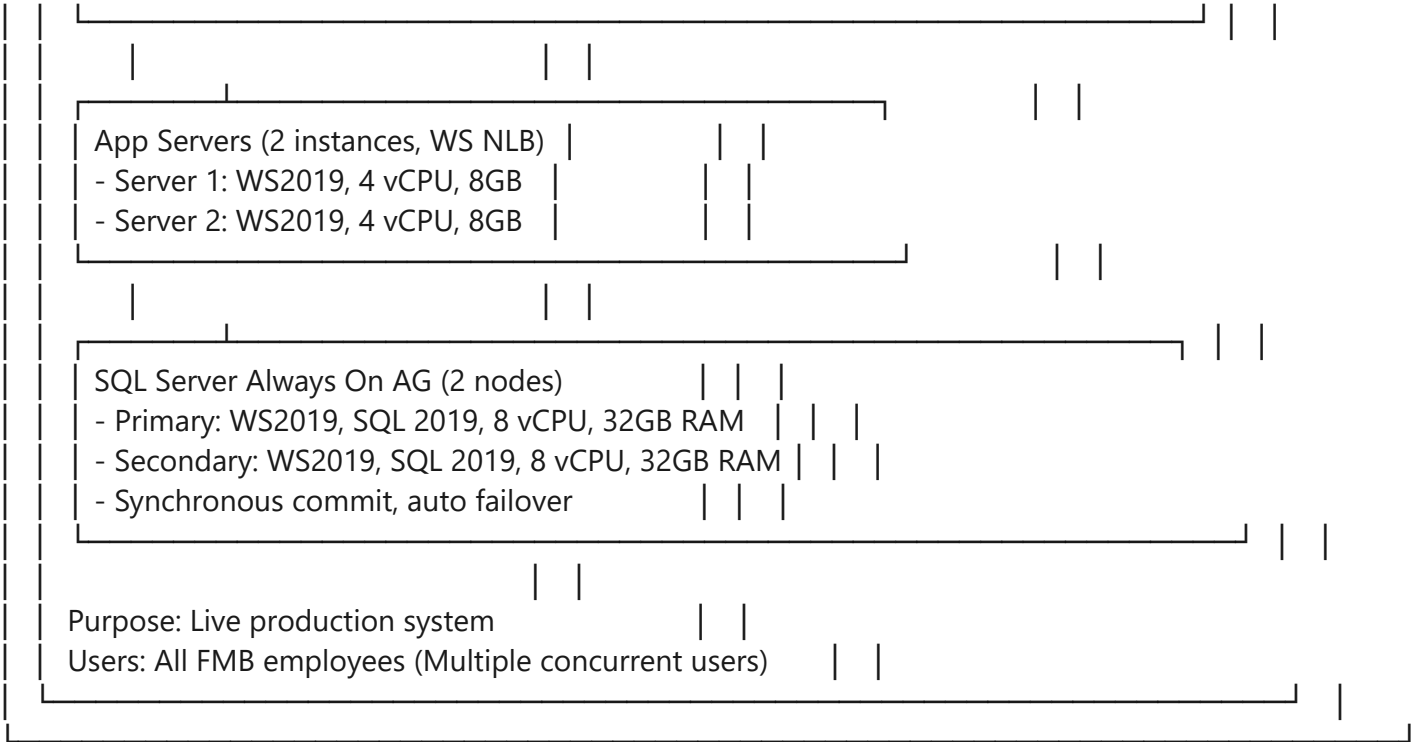
Authorization APIs: - GET /api/auth/permissions - Get user permissions - POST /api/auth/check-permission - Check conditional permission - GET /api/admin/users - List users (admin only) - POST /api/admin/users/:id/roles/manual - Assign role (admin only)

6. Infrastructure and Deployment Architecture

6.1 FMB Deployment Topology (Three Environments)

Environment Architecture:





Environment Specifications:

Environment	App Servers	SQL Server	Nginx	Purpose
Development	1 (4 vCPU, 8GB)	Standalone	No	Feature dev, integration testing
UAT	2 (4 vCPU, 8GB)	AG 2-node	Optional	User acceptance testing, pre-prod validation
Production	2 (4 vCPU, 8GB)	AG 2-node	HA pair	Live system, high availability SLA

6.2 Deployment Strategy for FMB

Deployment Process (Dev → UAT → Production):

- 1. Development Environment
 - o Developers commit code to develop branch
 - o Azure DevOps / Jenkins triggers build pipeline
 - o Automated tests run (unit, integration)

- o Deploy to dev environment for smoke testing

2. **UAT Environment**

- o Merge develop to uat branch after dev testing
- o Deploy to UAT environment
- o Business users perform acceptance testing
- o QA performs regression testing
- o Sign-off required from Product Owner

3. **Production Environment**

- o Merge uat to main branch after UAT sign-off
- o Create release tag (e.g., v1.2.0)
- o Production deployment window: Scheduled deployment window
- o Blue-green deployment strategy (zero-downtime)
- o Monitor metrics for 1 hour post-deployment

Deployment Steps (Production):

Step 1: Pre-deployment checks

- Verify SQL Server AG health (both nodes online)
- Verify Nginx reverse proxy health
- Backup current application version

Step 2: Database migrations (if any)

- Run SQL migration scripts on primary AG node
- Validate schema changes on secondary replica

Step 3: Deploy new application version

- Deploy to standby app server (blue)
- Start Node.js application on blue server
- Run smoke tests on blue server

Step 4: Traffic cutover

- Update Nginx config to route Limited traffic to blue
- Monitor error rates, response time for Short interval
- Gradually increase to gradual increase to full traffic

Step 5: Post-deployment validation

- Run automated smoke tests
- Verify health checks passing
- Monitor application metrics for 1 hour

Step 6: Decommission old version

- Keep green environment for Standard session duration (rollback window)
- After validation period, decommission green servers

6.3 CI/CD Pipeline for FMB (Azure DevOps / Jenkins)

Build Pipeline:

Azure DevOps Pipeline Example

trigger:

branches:

include:

- develop
- uat
- main

pool:

name: 'FMB-OnPrem-Agents' *# On-premise build agents*

stages:

- stage: Build

jobs:

- job: BuildAndTest

steps:

- task: NodeTool@0

inputs:

versionSpec: '20.x'

- script: npm install

displayName: 'Install dependencies'

- script: npm run lint

displayName: 'Run linter'

- script: npm run check

displayName: 'TypeScript type checking'

- script: npm run test

displayName: 'Run unit tests'

- script: npm run build

displayName: 'Build frontend'

- task: CopyFiles@2

inputs:

contents: '**'

targetFolder: '\$(Build.ArtifactStagingDirectory)'

- task: PublishBuildArtifacts@1
 - inputs:
 - pathToPublish: '\$(Build.ArtifactStagingDirectory)'
 - artifactName: 'client360-app'

- stage: DeployDev
 - condition: eq(variables['Build.SourceBranch'], 'refs/heads/develop')
 - jobs:
 - deployment: DeployToDev
 - environment: 'Development'
 - strategy:
 - runOnce:
 - deploy:
 - steps:
 - script: |
 - xcopy /E /Y "\$(Pipeline.Workspace)\client360-app*" "\\dev-app01\c\$\client360\"
 - sc.exe \\dev-app01 stop "Client360Service"
 - sc.exe \\dev-app01 start "Client360Service"
 - displayName: 'Deploy to Dev Server'

 - stage: DeployUAT
 - condition: eq(variables['Build.SourceBranch'], 'refs/heads/uat')
 - jobs:
 - deployment: DeployToUAT
 - environment: 'UAT'
 - strategy:
 - runOnce:
 - deploy:
 - steps:
 - script: |
 - # Deploy to UAT servers (2 app servers)
 - xcopy /E /Y "\$(Pipeline.Workspace)\client360-app*" "\\uat-app01\c\$\client360\"
 - xcopy /E /Y "\$(Pipeline.Workspace)\client360-app*" "\\uat-app02\c\$\client360\"
 - # Restart services
 - sc.exe \\uat-app01 restart "Client360Service"
 - sc.exe \\uat-app02 restart "Client360Service"
 - displayName: 'Deploy to UAT Servers'

 - stage: DeployProduction
 - condition: eq(variables['Build.SourceBranch'], 'refs/heads/main')
 - jobs:
 - deployment: DeployToProduction
 - environment: 'Production'
 - strategy:
 - runOnce:
 - deploy:
 - steps:
 - script: |
 - # Blue-green deployment (deploy to standby server first)

```
xcopy /E /Y "$(Pipeline.Workspace)\client360-app\*" "\\prod-app02\c$\client360\"
sc.exe \\prod-app02 restart "Client360Service"
# Wait for smoke tests
timeout /t 60
# Update Nginx config to route traffic
displayName: 'Deploy to Production (Blue-Green)'
```

Quality Gates: - All tests passing (unit, integration) - Strong code coverage - No critical security vulnerabilities (npm audit) - TypeScript compilation successful - SQL migrations tested on UAT

7. Cross-Cutting Technical Capabilities

7.1 Security Architecture for FMB

Authentication Flow (RSA SAML 2.0 SSO):

1. User accesses <https://client360.fmb.internal> → no session → redirect to </saml/login>
2. App generates SAML AuthnRequest → POST to RSA IdP
3. User authenticates at RSA IdP (username/password + MFA)
4. IdP generates SAML Response → POST to <https://client360.fmb.internal/saml/acs>
5. App validates SAML assertion:
 - Signature verification (X.509 certificate from RSA)
 - Audience restriction (must match FMB SP entity ID)
 - Timestamp validation (NotBefore, NotOnOrAfter)
 - Assertion expiry (Short timeframe)
6. App extracts attributes:
 - Employee ID (required)
 - Email, name, department (optional)
 - SAML role (e.g., 'System_Admin', 'Branch_Manager')
7. App looks up/creates employee record in SQL Server
8. App processes SAML role → auto-assign application role (RBAC)
9. App creates session in SQL Server session store (24-hour TTL)
10. App redirects to home page with secure session cookie

RSA SAML Configuration:

```
<!-- Service Provider Metadata (client360.fmb.internal) -->
<EntityDescriptor entityID="https://client360.fmb.internal">
  <SPSSODescriptor>
    <KeyDescriptor use="signing">
      <X509Certificate>MIIDXTCCAkWgAwIBAgIJAK...</X509Certificate>
    </KeyDescriptor>
    <AssertionConsumerService
      Binding="urn:oasis:names:tc:SAML:2.0:bindings:HTTP-POST"
      Location="https://client360.fmb.internal/saml/acs"
      index="1"/>
    </SPSSODescriptor>
  </EntityDescriptor>
```

Authorization Flow (RBAC + ABAC):

1. User makes API request → session cookie included
2. Authentication middleware validates session:
 - Check SQL Server session table (sessionId → employeeId)
 - Validate expiry (Standard session duration)
3. Authorization middleware checks permission:
 - Call permissionService.checkPermission(employeeId, permissionCode)
 - Evaluate RBAC: Does user's role grant this permission?
 - Evaluate ABAC: Do attributes match conditions?
Example: Branch restriction (employee.branchId = customer.branchId)
4. If denied:
 - Log denial to audit log (employee, permission, reason, resource)
 - Return HTTP 403 Forbidden
5. If granted:
 - Proceed to business logic
 - Log action to audit log (employee, action, resource, timestamp)

Data Protection:

| Data State | Protection Mechanism | FMB Implementation |
|-------------------|----------------------|--|
| At Rest | SQL Server TDE | Database files encrypted (AES-256) |
| In Transit | TLS 1.3 | Nginx reverse proxy, all HTTPS |
| In Use | PII masking | SSN: *--1234, account numbers never logged |
| In Logs | PII redaction | Custom logger redacts SSN, passwords |

Audit Logging:

All user actions and security events are logged to SQL Server audit tables:

```
CREATE TABLE audit_log (  
  audit_id BIGINT IDENTITY(1,1) PRIMARY KEY,  
  event_timestamp DATETIME2 DEFAULT GETUTCDATE(),  
  event_type NVARCHAR(50), -- 'PERMISSION_DENIED', 'CUSTOMER_VIEW', 'NOTE_CREATED'  
  employee_id BIGINT,  
  employee_name NVARCHAR(200),  
  permission_code NVARCHAR(100),  
  resource_type NVARCHAR(50),  
  resource_id NVARCHAR(50),  
  denial_reason NVARCHAR(MAX),  
  ip_address NVARCHAR(50),
```



```
user_agent NVARCHAR(500),
session_id NVARCHAR(100)
);
```

-- Index for fast lookups

```
CREATE INDEX idx_audit_employee ON audit_log(employee_id, event_timestamp);
CREATE INDEX idx_audit_event_type ON audit_log(event_type, event_timestamp);
```

Log Retention: 7 years (regulatory requirement), indexed for search.

7.2 Monitoring and Observability

Application Metrics:

Key performance indicators monitored in production: - **Request Metrics:** HTTP request rate, error rate, latency (p50, p95, p99) - **Database Metrics:** Query execution time, connection pool usage, Always On AG status - **Session Metrics:** Active sessions, session creation/destruction rate - **Security Metrics:** Failed login attempts, permission denials, suspicious activity - **Business Metrics:** Customer searches per hour, notes created, households viewed

Health Checks:

```
// Health check endpoint
app.get('/health', async (req, res) => {
  const health = {
    status: 'healthy',
    timestamp: new Date().toISOString(),
    checks: {
      database: await checkDatabaseConnection(),
      sessions: await checkSessionStore(),
      api: 'ok'
    }
  };

  const isHealthy = Object.values(health.checks).every(c => c === 'ok');
  res.status(isHealthy ? 200 : 503).json(health);
});
```

Logging:

- **Application Logs:** Winston logger with structured JSON output
- **Audit Logs:** SQL Server audit tables for notes and role management (7-year retention)
- **System Logs:** Windows Event Log integration

7.3 Backup and Data Protection

Backup Strategy: - **SQL Server Backups:** Regular full, differential, and transaction log backups - **Retention:** Short-term online, mid-term nearline, long-term archival (compliance) - **Backup Validation:** Monthly restore test to dev environment

SQL Server Always On Availability Groups: - **Primary Node:** Read/write operations - **Secondary Node:** Asynchronous replication, read-only queries - **Automatic Failover:** Fast failover time - **Health Monitoring:** Automatic health detection and failover
